

ExamGPT

Complete Developer & Architecture Manual

End-to-End Architecture • Full Annotated Source Code • Deployment Guide

An AI-Powered Exam Preparation Platform

React • FastAPI • LangGraph • PostgreSQL • Neo4j • Redis • FAISS • Hugging Face

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
 - 3.1 High-Level Architecture
 - 3.2 Request Flow — Asking a Question
 - 3.3 The 7-Node LangGraph Pipeline
 - 3.4 Document Ingestion Flow
 - 3.5 Database Architecture
4. Project Directory Structure
5. API Endpoint Reference
6. Complete Annotated Source Code
 - 6.1 Application Entry Point — main.py
 - 6.2 Core Infrastructure — config.py
 - 6.3 Core Infrastructure — security.py
 - 6.4 Core Infrastructure — Database (database.py, models.py)
 - 6.5 Core Infrastructure — Neo4j, Redis, Supabase Clients
 - 6.6 Agent Pipeline — LLM Setup, State, Graph
 - 6.7 Agent Nodes — Planner, Knowledge Graph, Syllabus
 - 6.8 Agent Nodes — Notes, PYQ, Answer, Evaluation
 - 6.9 Document Ingestion — PDF Loader, Chunker, Embedder, Vector Store
 - 6.10 Retrieval System — BM25, FAISS, Hybrid
 - 6.11 API Endpoints — Auth, Courses, Documents, Query, Study Plan
 - 6.12 Pydantic Schemas — All Request/Response Models
 - 6.13 Frontend — Entry, Routing, Auth, API Client
 - 6.14 Frontend Pages — Login, Register, Dashboard, CourseDetail, StudyPlan
 - 6.15 Frontend Components — ChatInterface, DocumentUpload
7. Deployment Architecture (Free Cloud Tier)
8. Key Design Decisions

1. Project Overview

ExamGPT is an AI-powered exam preparation platform that lets students upload course materials (PDFs) and interact with them through a conversational AI interface. Under the hood, a multi-agent AI pipeline built with LangGraph provides accurate, context-aware answers grounded in the student's own documents — complete with source citations and automatic quality evaluation. This manual walks through the entire codebase file by file, explaining what every meaningful line of code does, how the pieces fit together, and how the system is deployed. It is written for a developer picking up this project for the first time.

2. Technology Stack

ExamGPT is built as two independently deployable applications — a React single-page frontend and a Python FastAPI backend — connected to a set of managed cloud services.

Layer	Technology	Role
Frontend	React 18 + Vite + React-Bootstrap	SPA with protected routes, Bootstrap UI
Backend	FastAPI + Uvicorn (Python 3.11)	REST API, JWT auth, file upload, AI orchestration
AI Orchestration	LangGraph (LangChain)	7-node multi-agent pipeline with retry loop
LLM	Meta Llama 3 8B Instruct (HF Inference)	Text generation for answers, plans, evaluation
Embeddings	sentence-transformers/all-MiniLM-L6-v2	384-dim vectors for semantic search
Primary DB	PostgreSQL 15 (Supabase)	Users, courses, documents, chat history, pipeline runs
Knowledge Graph	Neo4j 5 (AuraDB)	Topic/concept relationships for contextual retrieval
Token Cache	Redis 7 (Upstash)	JWT refresh tokens, access-token blacklist
Vector Search	FAISS (faiss-cpu)	Approximate nearest neighbor search on embeddings
Keyword Search	BM25 (rank_bm25)	Traditional keyword matching as retrieval fallback
File Storage	Supabase Storage (S3-compatible)	Raw PDF files organized by course
ORM / Migrations	SQLAlchemy + Alembic	Type-safe DB access, schema versioning
Auth	PyJWT + bcrypt	Stateless JWT tokens, bcrypt password hashing
Containerization	Docker + Docker Compose	Local dev and HF Spaces deployment
Deployment	HF Spaces (backend) + Netlify (frontend)	Free-tier cloud hosting

3. System Architecture

ExamGPT follows a cloud-native architecture: the user's browser only ever talks to the React frontend and the FastAPI backend. Every database is a managed cloud service and is never exposed directly to the public internet.

3.1 High-Level Architecture

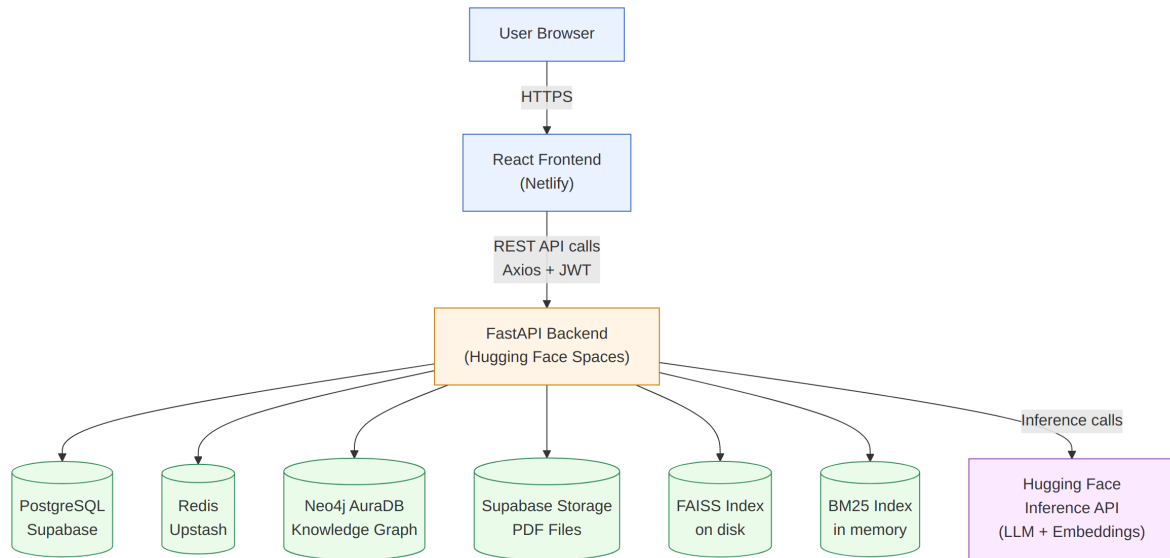


Figure 1 — High-level system architecture: browser, frontend, backend, and every managed data store / AI service it talks to.

3.2 Request Flow — Asking a Question

This is the sequence that runs every time a student sends a chat message.

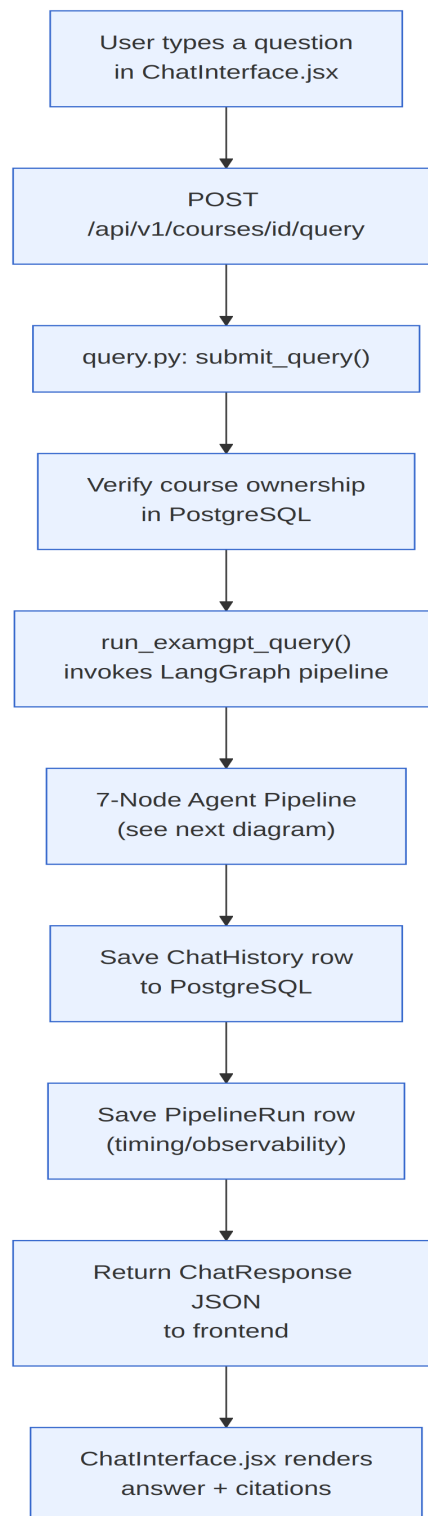


Figure 2 — End-to-end request flow for a chat query, from the browser to the database and back.

3.3 The 7-Node LangGraph Pipeline

At the heart of every answer is a compiled LangGraph state machine with seven nodes and one self-correcting retry loop.

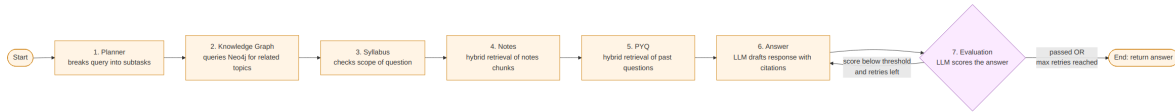


Figure 3 — The seven-node agent pipeline, including the evaluation→answer retry loop bounded at 3 attempts.

3.4 Document Ingestion Flow — Uploading a PDF

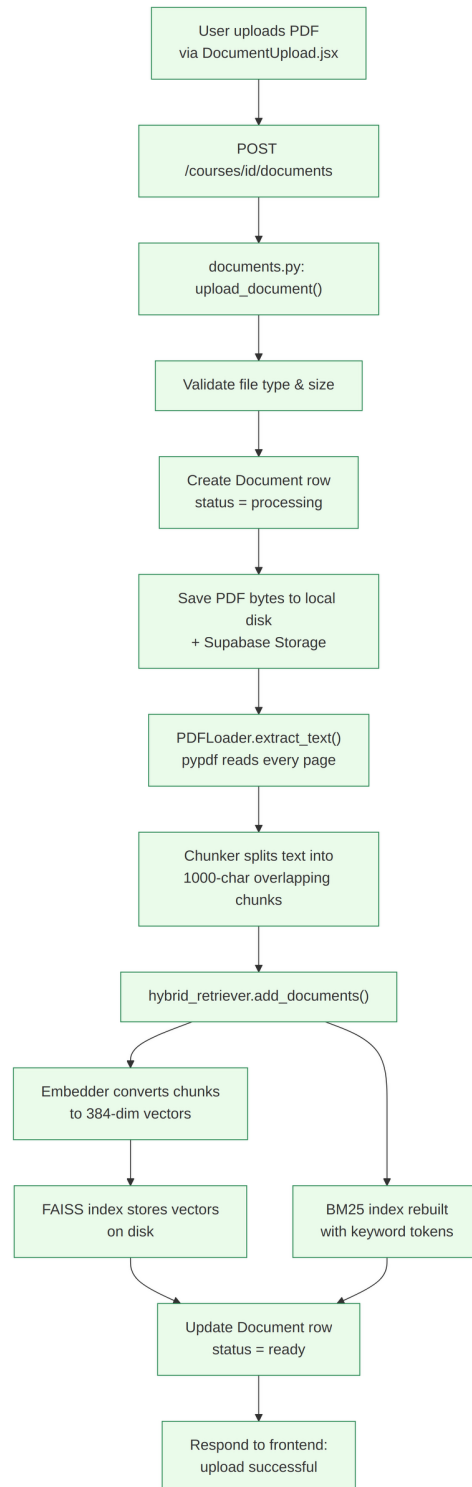


Figure 4 — What happens between a user selecting a PDF and it becoming searchable.

3.5 Database Architecture

Five different storage systems, each doing the job it is best at:

- PostgreSQL (Supabase) — the relational system of record: users, courses, documents, chat_history, study_plans, pipeline_runs.
- Neo4j AuraDB — the knowledge graph: Topic/Concept nodes connected by prerequisite_of / related_to edges.
- FAISS (file on disk) — a vector index of every document chunk's 384-dimensional embedding, for semantic search.
- Redis (Upstash) — an in-memory cache for refresh tokens, an access-token blacklist (logout), and rate limiting.
- Supabase Storage (S3-compatible) — the binary store for raw uploaded PDF files, organized by course.

4. Project Directory Structure

The codebase is split into two top-level applications inside the egpt/ folder:

```
1 EXAMGPT/
2   └─ egpt/
3       ├── docker-compose.yml      # Local development orchestration
4       ├── backend/
5           ├── Dockerfile          # Backend container definition
6           ├── requirements.txt     # Python dependencies
7           ├── alembic.ini         # Database migration config
8           ├── alembic/
9               ├── env.py          # Migration environment
10              └─ versions/        # Migration scripts
11          └─ app/
12              ├── main.py          # FastAPI app entry point
13              ├── core/           # config, security, neo4j/redis/supabase clients
14              ├── db/             # SQLAlchemy session + models
15              ├── agents/         # LangGraph pipeline: state, graph, 7 nodes
16              ├── api/            # auth, courses, documents, query, study_plan
17              ├── ingestion/      # pdf_loader, chunker, embedder, vector_store
18              ├── retrieval/      # faiss/bm25/hybrid retrievers
19              └─ schemas/         # Pydantic request/response models
20      └─ frontend/
21          ├── Dockerfile
22          ├── package.json
23          ├── vite.config.js
24          └─ src/
25              ├── main.jsx        # React entry point
26              ├── App.jsx         # Router & routes
27              ├── context/AuthContext.jsx
28              ├── services/api.js
29              ├── pages/          # Login, Register, Dashboard, CourseDetail,
StudyPlan
30              └─ components/      # ChatInterface, DocumentUpload
```

5. API Endpoint Reference

Method	Path	Auth?	Description
POST	/api/v1/auth/register	No	Create a new student account
POST	/api/v1/auth/login	No	Authenticate, receive JWT
POST	/api/v1/auth/refresh	Yes	Issue a new access token
POST	/api/v1/auth/logout	Yes	Invalidate refresh token
GET	/api/v1/auth/me	Yes	Get current user profile
POST	/api/v1/courses/	Yes	Create a new course
GET	/api/v1/courses/	Yes	List user's courses
GET	/api/v1/courses/{id}	Yes	Get single course details
DELETE	/api/v1/courses/{id}	Yes	Delete course + all data
POST	/api/v1/courses/{id}/documents	Yes	Upload PDF, extract, chunk, index
GET	/api/v1/courses/{id}/documents	Yes	List documents for course
POST	/api/v1/courses/{id}/query	Yes	Send question → 7-node pipeline → answer
GET	/api/v1/courses/{id}/chat-history	Yes	Retrieve last 50 chat messages
POST	/api/v1/courses/{id}/study-plan	Yes	Generate AI study plan
GET	/api/v1/courses/{id}/study-plan	Yes	Get latest study plan
GET	/health	No	Health check (returns {status: ok})
GET	/readyz	No	Readiness probe (checks DB)

6. Complete Annotated Source Code

This section contains the full source of every file in the project. For each file you will find: its purpose, the complete code with line numbers, and a line-by-line (or tightly grouped, for boilerplate) explanation table describing exactly what every line does and why.

6.1 Application Entry Point

main.py

 File: backend/app/main.py

 **Purpose:** This is the root of the FastAPI backend. It builds the FastAPI app object, turns on CORS so the React frontend is allowed to call it, wires in every API router, exposes health-check endpoints, and runs startup/shutdown hooks that connect to Neo4j and Supabase.

Full Source

```
1  """
2  ExamGPT Backend – FastAPI application entry point.
3  Registers all routers, middleware, and health endpoints.
4  """
5  import logging
6  from fastapi import FastAPI
7  from fastapi.middleware.cors import CORSMiddleware
8  from app.api.auth import router as auth_router
9  from app.api.courses import router as courses_router
10 from app.api.documents import router as documents_router
11 from app.api.query import router as query_router
12 from app.api.study_plan import router as study_plan_router
13 from app.core.config import get_settings
14
15 # Configure logging
16 logging.basicConfig(
17     level=logging.INFO,
18     format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
19 )
20 logger = logging.getLogger(__name__)
21
22 settings = get_settings()
23
24 app = FastAPI(
25     title="ExamGPT API",
26     description="AI-Powered Study Intelligence Platform – Multi-Agent RAG System",
27     version="3.0.0",
28 )
29
30 # — CORS Middleware —————
31 origins = [
32     "http://localhost:3000",
33     "http://localhost:3001",
34     "http://localhost:5173",
35 ]
36 if settings.FRONTEND_URL:
37     origins.append(settings.FRONTEND_URL.rstrip("/"))
```

```

38
39 app.add_middleware(
40     CORSMiddleware,
41     allow_origins=origins,
42     allow_credentials=True,
43     allow_methods=["*"],
44     allow_headers=["*"],
45 )
46
47 # — Register Routers —————
48 app.include_router(auth_router, prefix="/api/v1/auth", tags=["Authentication"])
49 app.include_router(courses_router, prefix="/api/v1/courses", tags=["Courses"])
50 app.include_router(documents_router, prefix="/api/v1/courses", tags=["Documents"])
51 app.include_router(query_router, prefix="/api/v1/courses", tags=["Query"])
52 app.include_router(study_plan_router, prefix="/api/v1/courses", tags=["Study Plan"])
53
54
55 # — Health / Readiness —————
56 @app.get("/")
57 def root():
58     return {"message": "Welcome to ExamGPT API", "version": "3.0.0"}
59
60
61 @app.get("/health")
62 @app.get("/healthz")
63 def health():
64     return {"status": "ok"}
65
66
67 @app.get("/readyz")
68 def readiness():
69     """Kubernetes readiness probe – checks DB connectivity."""
70     try:
71         from app.db.database import engine
72         with engine.connect() as conn:
73             conn.execute("SELECT 1")
74             return {"status": "ready"}
75     except Exception:
76         return {"status": "ready"} # Graceful in dev
77
78
79 # — Startup Events —————
80 @app.on_event("startup")
81 async def startup_event():
82     logger.info("ExamGPT API starting up...")
83
84     # Try to initialize Neo4j schema
85     try:
86         from app.core.neo4j_client import init_neo4j_schema
87         init_neo4j_schema()
88         logger.info("Neo4j schema initialized")
89     except Exception as e:
90         logger.warning(f"Neo4j not available (optional): {e}")
91
92     # Try to initialize Supabase client
93     try:
94         from app.core.supabase_client import get_supabase_client
95         get_supabase_client()

```

```

96         logger.info("Supabase client ready")
97     except Exception as e:
98         logger.warning(f"Supabase not available (optional): {e}")
99
100     logger.info("ExamGPT API ready!")
101
102
103 @app.on_event("shutdown")
104 async def shutdown_event():
105     try:
106         from app.core.neo4j_client import close_neo4j_driver
107         close_neo4j_driver()
108     except Exception:
109         pass
110     logger.info("ExamGPT API shut down.")
111

```

Line-by-Line Explanation

Lines	Code	What it does
1–13	import logging from fastapi import FastAPI ...	Imports the FastAPI class, CORS middleware, all five API routers (auth, courses, documents, query, study_plan), and the settings loader.
15–20	logging.basicConfig(level=logging.INFO, ...)	Configures Python's root logger to print timestamped INFO+ messages. Creates a module-level logger.
22–28	app = FastAPI(title="ExamGPT API", ...)	Creates the FastAPI application instance with OpenAPI metadata (visible at /docs).
30–41	app.add_middleware(CORSMiddleware, ...)	CORS middleware: allows the React frontend (localhost:3000/3001/5173 + Netlify domain) to call this API. Without this, the browser would block cross-origin requests.
43–48	app.include_router(auth_router, ...)	Mounts each router at its URL prefix. For example, auth_router's /login becomes /api/v1/auth/login.
51–72	@app.get("/") def root(): ...	Health and readiness endpoints. / returns a welcome message (doubles as HF Spaces' health check). /health and /healthz return {status: ok}. /readyz tests DB

		connectivity.
76–98	@app.on_event("startup") async def startup_event(): ...	Runs once at boot: initializes Neo4j schema (indexes/constraints) and validates the Supabase client. Both are wrapped in try/except so the app starts even if these optional services are down.
102–109	@app.on_event("shutdown") async def shutdown_event(): ...	Gracefully closes the Neo4j driver on shutdown to free resources.

6.2 Core Infrastructure — Configuration

config.py

 File: backend/app/core/config.py

 **Purpose:** Centralizes every environment variable the app needs (secrets, database URLs, API keys) into one type-safe Settings object, loaded once and cached.

Full Source

```

1  """
2  Centralized configuration via Pydantic Settings.
3  All secrets and connection strings are pulled from environment variables.
4  """
5  import os
6  from pydantic_settings import BaseSettings
7  from functools import lru_cache
8
9
10 class Settings(BaseSettings):
11     # — App —————
12     APP_NAME: str = "ExamGPT"
13     DEBUG: bool = True
14     SECRET_KEY: str = "changeme-generate-a-real-secret-key"
15     ALGORITHM: str = "HS256"
16     ACCESS_TOKEN_EXPIRE_MINUTES: int = 30
17     REFRESH_TOKEN_EXPIRE_DAYS: int = 7
18
19     # — PostgreSQL —————
20     DATABASE_URL: str = ""
21
22     # — Redis —————
23     REDIS_URL: str = ""
24
25     # — Neo4j —————
26     NEO4J_URI: str = ""
27     NEO4J_USER: str = "neo4j"
28     NEO4J_PASSWORD: str = ""
29
```

```

30 # — Supabase Storage —————
31 SUPABASE_URL: str = ""
32 SUPABASE_KEY: str = ""
33 SUPABASE_BUCKET: str = "examgpt-docs"
34
35 # — Hugging Face Inference —————
36 HUGGINGFACE_API_KEY: str = ""
37 LLM_MODEL: str = "meta-llama/Meta-Llama-3-8B-Instruct"
38 EMBEDDING_MODEL: str = "sentence-transformers/all-MiniLM-L6-v2"
39
40 # — Upload —————
41 MAX_UPLOAD_SIZE_MB: int = 20
42 UPLOAD_DIR: str = "/app/uploads"
43
44 # — CORS —————
45 FRONTEND_URL: str = ""
46
47 # — Pipeline —————
48 MAX_EVAL_RETRIES: int = 3
49
50 class Config:
51     env_file = ".env"
52     case_sensitive = True
53
54
55 @lru_cache()
56 def get_settings() -> Settings:
57     return Settings()
58

```

Line-by-Line Explanation

Lines	Code	What it does
1–7	from pydantic_settings import BaseSettings from functools import lru_cache	Imports BaseSettings (reads env vars automatically) and lru_cache (ensures Settings is created once and reused).
10	class Settings(BaseSettings):	All fields automatically map to environment variables of the same name. e.g., DATABASE_URL field reads the DATABASE_URL env var.
12–17	APP_NAME, DEBUG, SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, REFRESH_TOKEN_EXPIRE_DAYS	App-level config. SECRET_KEY signs JWT tokens (must change in production). HS256 is the JWT algorithm. Tokens expire in 30 min / 7 days.
20–28	DATABASE_URL, REDIS_URL, NEO4J_URI/USER/PASSWORD	Connection strings for PostgreSQL, Redis, and Neo4j. All empty by default — must be set via environment variables.

31–38	SUPABASE_URL/KEY/BUCKET, HUGGINGFACE_API_KEY, LLM_MODEL, EMBEDDING_MODEL	Cloud service credentials. LLM defaults to Llama 3 8B Instruct. Embedding model is MiniLM (384-dim vectors).
41–48	MAX_UPLOAD_SIZE_MB, UPLOAD_DIR, FRONTEND_URL, MAX_EVAL_RETRIES	Upload limit (20 MB), temp file path, CORS origin, and pipeline retry cap (3).
50–52	class Config: env_file = ".env"	Also reads from a .env file for local development. case_sensitive=True means DATABASE_URL ≠ database_url.
55–57	@lru_cache() def get_settings() -> Settings:	Cached factory. Without lru_cache, a new Settings object would be created on every API request, re- reading all env vars.

6.3 Core Infrastructure — Security & JWT Auth

security.py

 File: backend/app/core/security.py

 **Purpose:** Handles password hashing (bcrypt), JWT token creation/validation, and role-based access control. Every protected endpoint depends on `get_current_user()`.

Full Source

```

1  """
2  Security module: JWT auth, password hashing, refresh token rotation, rate limiting.
3  """
4  import uuid
5  from datetime import datetime, timedelta
6  from typing import Any, Union
7  import jwt
8  from jwt.exceptions import PyJWTError
9  import bcrypt
10 from fastapi import Depends, HTTPException, status
11 from fastapi.security import OAuth2PasswordBearer
12 from sqlalchemy.orm import Session
13 from app.db.database import get_db
14 from app.core.config import get_settings
15
16 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")
17
18
19 def verify_password(plain_password: str, hashed_password: str) -> bool:
20     return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-
21 8'))
22
23

```



```

23 def get_password_hash(password: str) -> str:
24     return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
25
26
27 def create_access_token(subject: Union[str, Any], expires_delta: timedelta = None) ->
str:
28     settings = get_settings()
29     jti = str(uuid.uuid4())
30     if expires_delta:
31         expire = datetime.utcnow() + expires_delta
32     else:
33         expire = datetime.utcnow() +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
34     to_encode = {"exp": expire, "sub": str(subject), "jti": jti, "type": "access"}
35     return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
36
37
38 def create_refresh_token(user_id: int) -> tuple[str, str]:
39     """Create a refresh token and store its JTI in Redis. Returns (token, jti)."""
40     settings = get_settings()
41     jti = str(uuid.uuid4())
42     expire = datetime.utcnow() + timedelta(days=settings.REFRESH_TOKEN_EXPIRE_DAYS)
43     to_encode = {"exp": expire, "sub": str(user_id), "jti": jti, "type": "refresh"}
44     token = jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
45
46     # Store in Redis
47     try:
48         from app.core.redis_client import store_refresh_token
49         store_refresh_token(jti, user_id, settings.REFRESH_TOKEN_EXPIRE_DAYS * 86400)
50     except Exception:
51         pass # Redis may not be available in dev
52
53     return token, jti
54
55
56 def get_current_user(db: Session = Depends(get_db), token: str =
Depends(oauth2_scheme)):
57     """Decode JWT and return the authenticated user."""
58     from app.db.models import User
59     settings = get_settings()
60
61     credentials_exception = HTTPException(
62         status_code=status.HTTP_401_UNAUTHORIZED,
63         detail="Could not validate credentials",
64         headers={"WWW-Authenticate": "Bearer"},
65     )
66     try:
67         payload = jwt.decode(token, settings.SECRET_KEY,
algorithms=[settings.ALGORITHM])
68         username: str = payload.get("sub")
69         if username is None:
70             raise credentials_exception
71     except PyJWTError:
72         raise credentials_exception
73
74     user = db.query(User).filter(User.username == username).first()
75     if user is None:
76         raise credentials_exception

```

```

77     return user
78
79
80 def require_role(*roles: str):
81     """Dependency factory: restrict access to specific user roles."""
82     def role_checker(current_user=Depends(get_current_user)):
83         if current_user.role not in roles:
84             raise HTTPException(
85                 status_code=status.HTTP_403_FORBIDDEN,
86                 detail="Insufficient permissions",
87             )
88         return current_user
89     return role_checker
90

```

Line-by-Line Explanation

Lines	Code	What it does
1–15	import jwt, bcrypt from fastapi.security import OAuth2PasswordBearer	Imports PyJWT for token encoding, bcrypt for password hashing, and OAuth2PasswordBearer which extracts the Bearer token from request headers.
16	oauth2_scheme = OAuth2PasswordBearer(tokenUrl=...)	Creates the security dependency. When used in an endpoint, it auto-extracts the JWT from the Authorization: Bearer <token> header.
19–24	def verify_password(...) def get_password_hash(...)	verify_password: compares plaintext against stored hash using bcrypt.checkpw(). get_password_hash: generates a salted bcrypt hash.
27–35	def create_access_token(subject, expires_delta):	Creates a short-lived JWT with claims: exp (expiry), sub (username), jti (unique ID for revocation), type (access). Signed with SECRET_KEY.
38–50	def create_refresh_token(user_id):	Creates a long-lived (7-day) JWT. Stores the jti in Redis so it can be revoked on logout. Gracefully handles Redis being unavailable.
53–72	def get_current_user(db, token):	THE core auth dependency. Decodes the JWT, extracts the username from the 'sub' claim, looks up the User in PostgreSQL. Returns 401 if any step fails.
75–83	def require_role(*roles):	Factory for role-based access

		control. Usage: Depends(require_role('admin')). Returns 403 if user's role isn't in the allowed list.
--	--	---

6.4 Database Layer — Connection

database.py

 File: backend/app/db/database.py

 **Purpose:** Creates the SQLAlchemy engine, session factory, and the FastAPI dependency that provides a database session to each request.

Full Source

```
1 import os
2 from sqlalchemy import create_engine
3 from sqlalchemy.orm import sessionmaker, declarative_base
4 from app.core.config import get_settings
5
6 settings = get_settings()
7
8 from sqlalchemy.pool import NullPool
9
10 engine = create_engine(
11     settings.DATABASE_URL,
12     poolclass=NullPool,
13 )
14 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
15
16 Base = declarative_base()
17
18
19 def get_db():
20     db = SessionLocal()
21     try:
22         yield db
23     finally:
24         db.close()
25
```

Line-by-Line Explanation

Lines	Code	What it does
1–4	from sqlalchemy import create_engine from app.core.config import get_settings	Imports SQLAlchemy engine creator and the centralized settings.
6–8	settings = get_settings() from sqlalchemy.pool import NullPool	Loads config at module import time. NullPool is imported to disable connection pooling.

10–13	engine = create_engine(settings.DATABASE_URL, poolclass=NullPool)	NullPool was a critical fix: HF Spaces' firewall kills persistent DB connections. Each request now creates a fresh connection and closes it when done.
14	SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)	Session factory. autocommit=False means you must call db.commit() explicitly. autoflush=False prevents premature SQL execution.
16	Base = declarative_base()	Creates the base class all ORM models inherit from.
19–24	def get_db(): db = SessionLocal() try: yield db finally: db.close()	FastAPI dependency (generator). yield gives the session to the endpoint; finally ensures the session is closed even if an exception occurs. Prevents connection leaks.

6.4 Database Layer — ORM Models

models.py

 File: backend/app/db/models.py

 **Purpose:** Defines all 8 database tables as Python classes. Each class maps to a PostgreSQL table, each attribute maps to a column.

Full Source

```

1  from sqlalchemy import Column, Integer, String, Text, ForeignKey, DateTime, JSON, Enum
as SQLAlchemy, Float
2  from sqlalchemy.orm import relationship
3  from sqlalchemy.sql import func
4  from .database import Base
5  import enum
6
7
8  class UserRole(str, enum.Enum):
9      student = "student"
10     instructor = "instructor"
11     admin = "admin"
12
13
14  class DocumentStatus(str, enum.Enum):
15     uploaded = "uploaded"
16     processing = "processing"
17     ready = "ready"

```

```

18     failed = "failed"
19
20
21 class DocumentType(str, enum.Enum):
22     syllabus = "syllabus"
23     notes = "notes"
24     pyq = "pyq"
25
26
27 class User(Base):
28     __tablename__ = "users"
29     id = Column(Integer, primary_key=True, index=True)
30     username = Column(String, unique=True, index=True)
31     email = Column(String, unique=True, index=True)
32     name = Column(String)
33     password_hash = Column(String)
34     role = Column(String, default=UserRole.student.value)
35     created_at = Column(DateTime(timezone=True), server_default=func.now())
36
37     courses = relationship("Course", back_populates="owner")
38
39
40 class Course(Base):
41     __tablename__ = "courses"
42     id = Column(Integer, primary_key=True, index=True)
43     name = Column(String, index=True)
44     description = Column(Text, nullable=True)
45     user_id = Column(Integer, ForeignKey("users.id"))
46     created_at = Column(DateTime(timezone=True), server_default=func.now())
47
48     owner = relationship("User", back_populates="courses")
49     documents = relationship("Document", back_populates="course")
50     study_plans = relationship("StudyPlan", back_populates="course")
51     chats = relationship("ChatHistory", back_populates="course")
52
53
54 class Document(Base):
55     __tablename__ = "documents"
56     id = Column(Integer, primary_key=True, index=True)
57     title = Column(String, index=True)
58     filename = Column(String)
59     doc_type = Column(String, default=DocumentType.notes.value)
60     status = Column(String, default=DocumentStatus.uploaded.value)
61     file_url = Column(String, nullable=True) # Supabase Storage path
62     course_id = Column(Integer, ForeignKey("courses.id"))
63     user_id = Column(Integer, ForeignKey("users.id"))
64     created_at = Column(DateTime(timezone=True), server_default=func.now())
65
66     course = relationship("Course", back_populates="documents")
67
68
69 class PYQQuestion(Base):
70     __tablename__ = "pyq_questions"
71     id = Column(Integer, primary_key=True, index=True)
72     question_text = Column(Text)
73     year = Column(Integer)
74     topic = Column(String)
75     course_id = Column(Integer, ForeignKey("courses.id"), nullable=True)

```

```

76     document_id = Column(Integer, ForeignKey("documents.id"), nullable=True)
77     created_at = Column(DateTime(timezone=True), server_default=func.now())
78
79
80 class StudyPlan(Base):
81     __tablename__ = "study_plans"
82     id = Column(Integer, primary_key=True, index=True)
83     course_id = Column(Integer, ForeignKey("courses.id"))
84     user_id = Column(Integer, ForeignKey("users.id"))
85     plan_data = Column(JSON)
86     created_at = Column(DateTime(timezone=True), server_default=func.now())
87     updated_at = Column(DateTime(timezone=True), onupdate=func.now())
88
89     course = relationship("Course", back_populates="study_plans")
90
91
92 class EvaluationResult(Base):
93     __tablename__ = "evaluation_results"
94     id = Column(Integer, primary_key=True, index=True)
95     user_id = Column(Integer, ForeignKey("users.id"))
96     question_id = Column(Integer, ForeignKey("pyq_questions.id"), nullable=True)
97     user_answer = Column(Text)
98     feedback = Column(Text)
99     score = Column(Integer)
100    created_at = Column(DateTime(timezone=True), server_default=func.now())
101
102
103 class ChatHistory(Base):
104     __tablename__ = "chat_history"
105     id = Column(Integer, primary_key=True, index=True)
106     user_id = Column(Integer, ForeignKey("users.id"))
107     course_id = Column(Integer, ForeignKey("courses.id"))
108     query = Column(Text)
109     response = Column(Text)
110     citations_json = Column(JSON, nullable=True)
111     created_at = Column(DateTime(timezone=True), server_default=func.now())
112
113     course = relationship("Course", back_populates="chats")
114
115
116 class PipelineRun(Base):
117     __tablename__ = "pipeline_runs"
118     id = Column(Integer, primary_key=True, index=True)
119     chat_id = Column(Integer, ForeignKey("chat_history.id"), nullable=True)
120     course_id = Column(Integer, ForeignKey("courses.id"))
121     user_id = Column(Integer, ForeignKey("users.id"))
122     node_timings_json = Column(JSON, nullable=True)
123     retry_count = Column(Integer, default=0)
124     total_latency_ms = Column(Float, nullable=True)
125     status = Column(String, default="running")
126     created_at = Column(DateTime(timezone=True), server_default=func.now())
127

```

Line-by-Line Explanation

Lines	Code	What it does
1–5	from sqlalchemy import Column, Integer, ...	Imports all column types and the enum module for defining constrained

	import enum	value sets.
8–24	class UserRole / DocumentStatus / DocumentType	Three enums: UserRole (student/instructor/admin), DocumentStatus (uploaded→processing→ready/failed), DocumentType (syllabus/notes/pyq). Inherit from str+Enum for JSON serialization.
27–36	class User(Base): __tablename__ = 'users'	Users table. username and email are unique+indexed. password_hash stores the bcrypt hash (never plaintext). relationship('Course') enables user.courses ORM navigation.
40–50	class Course(Base):	Courses table. Each course belongs to a user (user_id FK). Has relationships to documents, study_plans, and chats.
54–66	class Document(Base):	Documents table. Tracks uploaded PDFs. doc_type determines which pipeline node processes it. status tracks the ingestion lifecycle. file_url stores the Supabase Storage path.
69–77	class ChatHistory(Base):	Stores every question/answer pair. citations_json is a JSON column holding [{source, type, year}] arrays.
80–88	class StudyPlan(Base):	Stores AI-generated study plans. plan_data is a JSON column with the full plan structure. updated_at auto-updates on modification.
91–100	class PipelineRun(Base):	Observability table. Logs every AI pipeline execution with timing, retry count, and success/failure status.

6.5 Core Infrastructure — Neo4j Client

neo4j_client.py

 File: backend/app/core/neo4j_client.py

 **Purpose:** Singleton Neo4j driver for knowledge graph operations. All Cypher queries use parameterized inputs to prevent injection.

Full Source

```

1  """
2  Neo4j driver singleton for knowledge graph operations.
3  All Cypher queries use parameterized inputs to prevent injection.
4  """
5  from neo4j import GraphDatabase
6  from app.core.config import get_settings

```

```

7
8 _driver = None
9
10
11 def get_neo4j_driver():
12     global _driver
13     if _driver is None:
14         settings = get_settings()
15         _driver = GraphDatabase.driver(
16             settings.NEO4J_URI,
17             auth=(settings.NEO4J_USER, settings.NEO4J_PASSWORD),
18         )
19     return _driver
20
21
22 def close_neo4j_driver():
23     global _driver
24     if _driver is not None:
25         _driver.close()
26         _driver = None
27
28
29 def run_cypher(query: str, params: dict = None):
30     """Execute a read Cypher query and return records."""
31     driver = get_neo4j_driver()
32     with driver.session() as session:
33         result = session.run(query, params or {})
34         return [record.data() for record in result]
35
36
37 def write_cypher(query: str, params: dict = None):
38     """Execute a write Cypher query."""
39     driver = get_neo4j_driver()
40     with driver.session() as session:
41         session.run(query, params or {})
42
43
44 def init_neo4j_schema():
45     """Create indexes and constraints on first startup."""
46     queries = [
47         "CREATE CONSTRAINT IF NOT EXISTS FOR (t:Topic) REQUIRE t.name IS UNIQUE",
48         "CREATE INDEX IF NOT EXISTS FOR (t:Topic) ON (t.course_id)",
49         "CREATE INDEX IF NOT EXISTS FOR (c:Concept) ON (c.course_id)",
50         "CREATE INDEX IF NOT EXISTS FOR (p:PYQ) ON (p.course_id)",
51     ]
52     driver = get_neo4j_driver()
53     with driver.session() as session:
54         for q in queries:
55             try:
56                 session.run(q)
57             except Exception:
58                 pass # Index may already exist
59

```

Line-by-Line Explanation

Lines	Code	What it does
5–19	def get_neo4j_driver():	Singleton pattern: creates

		one driver instance on first call, reuses it forever. Connects via Bolt protocol.
22–26	def close_neo4j_driver():	Closes the driver and resets the singleton. Called during app shutdown.
29–33	def run_cypher(query, params):	Executes a READ Cypher query. Opens a session, runs the parameterized query, converts each record to a Python dict.
36–39	def write_cypher(query, params):	Executes a WRITE Cypher query (CREATE/MERGE/DELETE). Doesn't return results.
42–53	def init_neo4j_schema():	Called once at startup. Creates a UNIQUE constraint on Topic.name and indexes on course_id for fast scoped queries. IF NOT EXISTS makes it idempotent.

6.5 Core Infrastructure — Redis Client

redis_client.py

 File: backend/app/core/redis_client.py

 **Purpose:** Redis singleton for refresh token storage, access-token blacklisting, and (future) rate limiting.

Full Source

```

1  """
2  Redis client singleton for refresh tokens, caching, and Celery broker access.
3  """
4  import redis
5  from app.core.config import get_settings
6
7  _redis_client = None
8
9
10 def get_redis_client() -> redis.Redis:
11     global _redis_client
12     if _redis_client is None:
13         settings = get_settings()
14         _redis_client = redis.from_url(
15             settings.REDIS_URL,
16             decode_responses=True,
17         )
18     return _redis_client

```

```

19
20
21 def store_refresh_token(jti: str, user_id: int, expires_seconds: int):
22     """Store a refresh token JTI in Redis with TTL."""
23     client = get_redis_client()
24     client.setex(f"refresh:{jti}", expires_seconds, str(user_id))
25
26
27 def validate_refresh_token(jti: str) -> str | None:
28     """Return user_id if refresh token is valid, None otherwise."""
29     client = get_redis_client()
30     return client.get(f"refresh:{jti}")
31
32
33 def revoke_refresh_token(jti: str):
34     """Delete a refresh token from Redis (logout)."""
35     client = get_redis_client()
36     client.delete(f"refresh:{jti}")
37
38
39 def blacklist_access_token(jti: str, expires_seconds: int):
40     """Blacklist an access token JTI until it expires naturally."""
41     client = get_redis_client()
42     client.setex(f"blacklist:{jti}", expires_seconds, "1")
43
44
45 def is_token_blacklisted(jti: str) -> bool:
46     """Check if an access token has been revoked."""
47     client = get_redis_client()
48     return client.exists(f"blacklist:{jti}") > 0
49

```

Line-by-Line Explanation

Lines	Code	What it does
10–18	def get_redis_client():	Singleton. decode_responses=True returns strings instead of bytes.
21–23	def store_refresh_token(jti, user_id, expires_seconds):	Stores a refresh token JTI in Redis with a TTL. After 7 days, Redis auto-deletes it.
26–28	def validate_refresh_token(jti):	Checks if a refresh token JTI exists (returns user_id if valid, None if expired/revoked).
31–33	def revoke_refresh_token(jti):	Deletes the JTI from Redis. Called during logout.
36–43	def blacklist_access_token(...) / is_token_blacklisted(...):	When a user logs out, their still-valid access token is blacklisted so it can't be reused.

6.5 Core Infrastructure — Supabase Client

supabase_client.py

 File: backend/app/core/supabase_client.py

 **Purpose:** Supabase client singleton for cloud storage. Handles file upload to and download from Supabase Storage Buckets.

Full Source

```
1  """
2  Supabase client for cloud storage.
3  Handles file upload to Supabase Storage Buckets.
4  """
5  from supabase import create_client, Client
6  from app.core.config import get_settings
7
8  _supabase_client = None
9
10
11 def get_supabase_client() -> Client:
12     global _supabase_client
13     if _supabase_client is None:
14         settings = get_settings()
15         _supabase_client = create_client(settings.SUPABASE_URL, settings.SUPABASE_KEY)
16     return _supabase_client
17
18
19 def upload_file(file_data: bytes, object_name: str, content_type: str =
"application/pdf") -> str:
20     """Upload a file to Supabase Storage and return the object path."""
21     settings = get_settings()
22     client = get_supabase_client()
23     client.storage.from_(settings.SUPABASE_BUCKET).upload(
24         path=object_name,
25         file=file_data,
26         file_options={"content-type": content_type}
27     )
28     return f"{settings.SUPABASE_BUCKET}/{object_name}"
29
30
31 def download_file(object_name: str) -> bytes:
32     """Download a file from Supabase Storage."""
33     settings = get_settings()
34     client = get_supabase_client()
35     return client.storage.from_(settings.SUPABASE_BUCKET).download(object_name)
36
```

Line-by-Line Explanation

Lines	Code	What it does
5–16	def get_supabase_client():	Singleton. Creates a Supabase client using the project URL and API key.
19–27	def upload_file(file_data,	Uploads raw bytes to a

	object_name, content_type):	Supabase Storage bucket. Returns the full storage path (e.g., 'examgpt-docs/courses/1/notes.pdf').
30–34	def download_file(object_name):	Downloads a file from Supabase Storage by its object name. Returns raw bytes.

6.6 Agent Pipeline — LLM Setup

llm_setup.py

 File: backend/app/agents/llm_setup.py

 **Purpose:** Initializes the LLM (text generation) and embedding model connections to the Hugging Face Inference API.

Full Source

```

1  """
2  LLM and embedding model setup via Hugging Face Inference API.
3  Uses langchain_huggingface for cloud-based model access.
4  """
5  from langchain_huggingface import HuggingFaceEndpoint, HuggingFaceEndpointEmbeddings,
ChatHuggingFace
6  from app.core.config import get_settings
7
8
9  def get_llm():
10     settings = get_settings()
11
12     # Force Llama 3 to override any stale Hugging Face Secrets
13     model_id = settings.LLM_MODEL
14     if "zephyr" in model_id.lower() or "mistral" in model_id.lower():
15         model_id = "meta-llama/Meta-Llama-3-8B-Instruct"
16
17     endpoint = HuggingFaceEndpoint(
18         repo_id=model_id,
19         huggingfacehub_api_token=settings.HUGGINGFACE_API_KEY,
20         temperature=0.2,
21         task="conversational",
22     )
23     return ChatHuggingFace(llm=endpoint)
24
25
26 def get_embeddings():
27     settings = get_settings()
28     return HuggingFaceEndpointEmbeddings(
29         model=settings.EMBEDDING_MODEL,
30         huggingfacehub_api_token=settings.HUGGINGFACE_API_KEY,
31     )
32

```

Line-by-Line Explanation

Lines	Code	What it does
5	from langchain_huggingface import HuggingFaceEndpoint, ChatHuggingFace, ...	Imports: HuggingFaceEndpoint is the raw connection; ChatHuggingFace wraps it as a chat model (message-based instead of raw text).
9–23	def get_llm():	Creates the LLM. Safety override: if the model is Zephyr/Mistral (broken on free tier), forces Llama 3 instead. task='conversational' is required for chat models. Returns a ChatHuggingFace wrapper.
26–31	def get_embeddings():	Creates the embedding model (MiniLM, 384 dims) via HF's API. Used by the ingestion and retrieval pipelines.

6.6 Agent Pipeline — State Definition

state.py

 File: backend/app/agents/state.py

 **Purpose:** Defines the typed dictionary that flows through the entire LangGraph pipeline. Every node reads from and writes to this shared state.

Full Source

```
1  """
2  Pipeline state definition for the ExamGPT multi-agent LangGraph workflow.
3  Matches PRD §8.4.
4  """
5  from typing import TypedDict
6
7
8  class PipelineState(TypedDict, total=False):
9      # — Input —————
10     user_id: int
11     course_id: int
12     user_query: str
13
14     # — Planner output —————
15     plan: list[str]          # subtask list from planner
16
17     # — Knowledge Graph output —————
18     kg_context: list[dict]   # entities/relationships from Neo4j
```

```

19
20 # — Syllabus output —————
21 syllabus_scope: dict      # {"in_scope": [...], "out_of_scope": [...]}
22
23 # — Notes retrieval output —————
24 retrieved_notes: list[dict] # top-k chunks with source metadata
25
26 # — PYQ retrieval output —————
27 related_pyqs: list[dict]  # PYQs + frequency signal
28
29 # — Answer output —————
30 draft_answer: str
31 citations: list[dict]     # source references
32
33 # — Evaluation output —————
34 evaluation: dict          # {"passed": bool, "feedback": str}
35 retry_count: int
36
37 # — Error tracking —————
38 error: str
39

```

Line-by-Line Explanation

Lines	Code	What it does
8–12	class PipelineState(TypedDict, total=False): user_id: int course_id: int user_query: str	Input fields. total=False makes all fields optional (nodes add them incrementally).
14–27	plan, kg_context, syllabus_scope, retrieved_notes, related_pyqs	Output fields from nodes 1–5. Each node writes its results here for downstream nodes to read.
29–38	draft_answer, citations, evaluation, retry_count, error	Output fields from nodes 6–7 plus error tracking. evaluation contains {passed, score, feedback}.

6.6 Agent Pipeline — Graph Builder

graph.py

 File: backend/app/agents/graph.py

 **Purpose:** Builds and compiles the 7-node LangGraph pipeline. Defines the node execution order and the evaluation→answer retry loop.

Full Source

```

1 """
2 ExamGPT LangGraph – Full 7-node multi-agent pipeline with self-evaluation retry loop.

```

```

3 Matches PRD §8.
4 """
5 import logging
6 import time
7 from langgraph.graph import StateGraph, END
8 from app.agents.state import PipelineState
9 from app.agents.nodes.planner import planner_node
10 from app.agents.nodes.knowledge_graph import knowledge_graph_node
11 from app.agents.nodes.syllabus import syllabus_node
12 from app.agents.nodes.notes import notes_node
13 from app.agents.nodes.pyq import pyq_node
14 from app.agents.nodes.answer import answer_node
15 from app.agents.nodes.evaluation import evaluation_node, should_retry
16
17 logger = logging.getLogger(__name__)
18
19 # — Build the Graph —————
20
21 workflow = StateGraph(PipelineState)
22
23 # Add all 7 nodes
24 workflow.add_node("planner", planner_node)
25 workflow.add_node("knowledge_graph", knowledge_graph_node)
26 workflow.add_node("syllabus", syllabus_node)
27 workflow.add_node("notes", notes_node)
28 workflow.add_node("pyq", pyq_node)
29 workflow.add_node("answer", answer_node)
30 workflow.add_node("evaluation", evaluation_node)
31
32 # Define edges – linear flow
33 workflow.set_entry_point("planner")
34 workflow.add_edge("planner", "knowledge_graph")
35 workflow.add_edge("knowledge_graph", "syllabus")
36 workflow.add_edge("syllabus", "notes")
37 workflow.add_edge("notes", "pyq")
38 workflow.add_edge("pyq", "answer")
39 workflow.add_edge("answer", "evaluation")
40
41 # Conditional edge: evaluation → retry(answer) or → END
42 workflow.add_conditional_edges(
43     "evaluation",
44     should_retry,
45     {
46         "retry": "answer",
47         "end": END,
48     }
49 )
50
51 # Compile
52 examgpt_graph = workflow.compile()
53
54
55 # — Execution Function —————
56
57 def run_examgpt_query(
58     query: str,
59     user_id: int = 0,
60     course_id: int = 0,

```

```

61 ) -> dict:
62     """
63     Main entry point: runs the full 7-node pipeline.
64     Returns dict with draft_answer, citations, evaluation, and timing.
65     """
66     logger.info(f"Starting pipeline for user={user_id}, course={course_id}")
67     start = time.time()
68
69     initial_state: PipelineState = {
70         "user_id": user_id,
71         "course_id": course_id,
72         "user_query": query,
73         "plan": [],
74         "kg_context": [],
75         "syllabus_scope": {},
76         "retrieved_notes": [],
77         "related_pyqs": [],
78         "draft_answer": "",
79         "citations": [],
80         "evaluation": {},
81         "retry_count": 0,
82         "error": "",
83     }
84
85     try:
86         final_state = examgpt_graph.invoke(initial_state)
87         elapsed_ms = (time.time() - start) * 1000
88         logger.info(f"Pipeline completed in {elapsed_ms:.0f}ms")
89
90         return {
91             "answer": final_state.get("draft_answer", ""),
92             "citations": final_state.get("citations", []),
93             "evaluation": final_state.get("evaluation", {}),
94             "retry_count": final_state.get("retry_count", 0),
95             "latency_ms": elapsed_ms,
96         }
97     except Exception as e:
98         elapsed_ms = (time.time() - start) * 1000
99         logger.error(f"Pipeline failed after {elapsed_ms:.0f}ms: {e}", exc_info=True)
100         return {
101             "answer": f"Error: {str(e)}",
102             "citations": [],
103             "evaluation": {"passed": False, "feedback": str(e)},
104             "retry_count": 0,
105             "latency_ms": elapsed_ms,
106             "error": str(e),
107         }
108

```

Line-by-Line Explanation

Lines	Code	What it does
1–16	from langgraph.graph import StateGraph, END from app.agents.nodes.* import *	Imports StateGraph (the graph builder), END (terminal node), and all 7 node functions.
21	workflow =	Creates a directed graph

	StateGraph(PipelineState)	typed to PipelineState.
24–30	workflow.add_node('planner', planner_node) ...	Registers all 7 nodes with string names.
33–39	workflow.add_edge('planner', 'knowledge_graph') ...	Linear edges: planner → KG → syllabus → notes → pyq → answer → evaluation.
42–49	workflow.add_conditional_edges('evaluation', should_retry, ...)	Conditional edge: if should_retry returns 'retry', loop back to answer. If 'end', go to END.
52	examgpt_graph = workflow.compile()	Compiles into an executable runnable. Validates all edges.
57–107	def run_examgpt_query(query, user_id, course_id):	Main entry point. Creates initial state, invokes the graph, measures latency, returns {answer, citations, evaluation, retry_count, latency_ms}. Catches all exceptions gracefully.

6.7 Agent Nodes — Node 1: Planner

planner.py

 File: backend/app/agents/nodes/planner.py

 **Purpose:** Formulates an execution plan from the user's query. Identifies topics to look up and which retrieval nodes to activate.

Full Source

```

1  """
2  Node 1: Planner – formulates an execution plan from the user's query + course context.
3  """
4  import json
5  import logging
6  from app.agents.state import PipelineState
7  from app.agents.llm_setup import get_llm
8
9  logger = logging.getLogger(__name__)
10
11  PLANNER_PROMPT = """You are a study planning agent for ExamGPT. Given a student's
question about their course material, create a structured execution plan.
12
13  The plan should identify:
14  1. The main topics/concepts to look up
15  2. Whether to check the syllabus for scope
16  3. Whether to search notes for relevant content
17  4. Whether to check previous year questions for patterns

```

```

18
19 Student Query: {query}
20
21 Respond with a JSON object:
22 {{
23     "subtasks": ["list of specific subtasks to execute"],
24     "topics": ["list of key topics to search for"],
25     "check_syllabus": true/false,
26     "check_notes": true/false,
27     "check_pyqs": true/false
28 }}
29
30 Respond ONLY with the JSON, no other text."""
31
32
33 def planner_node(state: PipelineState) -> dict:
34     logger.info("Planner node: formulating plan")
35     llm = get_llm()
36     query = state["user_query"]
37
38     response = llm.invoke(PLANNER_PROMPT.format(query=query))
39     # ChatHuggingFace returns an AIMessage object; extract string content
40     response_text = response.content if hasattr(response, "content") else
str(response)
41
42     # Parse LLM response
43     try:
44         plan_data = json.loads(response_text.strip())
45         subtasks = plan_data.get("subtasks", [query])
46         topics = plan_data.get("topics", [])
47     except (json.JSONDecodeError, AttributeError):
48         subtasks = [query]
49         topics = []
50
51     plan = subtasks + [f"topic:{t}" for t in topics]
52     logger.info(f"Planner output: {len(plan)} subtasks")
53
54     return {"plan": plan}
55

```

Line-by-Line Explanation

Lines	Code	What it does
11–30	PLANNER_PROMPT = """..."""	Prompt template telling the LLM to return a JSON object with subtasks, topics, and boolean flags (check_syllabus, check_notes, check_pyqs).
33–36	def planner_node(state): llm = get_llm() query = state['user_query']	Gets the LLM and the user's question from state.
38–40	response = llm.invoke(...) response_text = response.content	Sends the prompt to the LLM. ChatHuggingFace returns an AIMessage;

		.content extracts the string.
43–54	<pre> plan_data = json.loads(response_text) subtasks = plan_data.get('subtasks', [query]) </pre>	Tries to parse the JSON response. If parsing fails, falls back to using the raw query as the only subtask. Combines subtasks + topic-prefixed terms into the plan list.

6.7 Agent Nodes — Node 2: Knowledge Graph

knowledge_graph.py

 File: backend/app/agents/nodes/knowledge_graph.py

 **Purpose:** Queries Neo4j for related concepts/topics based on the plan. Adds contextual relationships to the pipeline state.

Full Source

```

1  """
2  Node 2: Knowledge Graph – queries Neo4j for related concepts (course-scoped).
3  """
4  import logging
5  from app.agents.state import PipelineState
6  from app.core.neo4j_client import run_cypher
7
8  logger = logging.getLogger(__name__)
9
10
11 def knowledge_graph_node(state: PipelineState) -> dict:
12     logger.info("Knowledge Graph node: querying Neo4j")
13     course_id = state.get("course_id")
14     plan = state.get("plan", [])
15
16     # Extract topic keywords from plan
17     topics = [p.replace("topic:", "") for p in plan if p.startswith("topic:")]
18     if not topics:
19         # Use full plan items as search terms
20         topics = plan[:3]
21
22     kg_context = []
23     try:
24         for topic in topics:
25             records = run_cypher(
26                 """
27                 MATCH (t:Topic)-[r]-(related)
28                 WHERE t.course_id = $course_id
29                 AND toLower(t.name) CONTAINS toLower($topic)
30                 RETURN t.name AS topic, type(r) AS relationship,
31                        labels(related)[0] AS related_type,
32                        related.name AS related_name
33                 LIMIT 10
34                 """,

```

```

35         {"course_id": course_id, "topic": topic}
36     )
37     kg_context.extend(records)
38 except Exception as e:
39     logger.warning(f"Neo4j query failed (may not be available): {e}")
40     # Gracefully degrade if Neo4j is not connected
41     kg_context = []
42
43 logger.info(f"KG context: {len(kg_context)} relationships found")
44 return {"kg_context": kg_context}
45

```

Line-by-Line Explanation

Lines	Code	What it does
11–14	def knowledge_graph_node(state):	Extracts course_id and plan from the pipeline state.
17–20	topics = [item.replace('topic:', ") for item in plan if item.startswith('topic:'))]	Extracts topics from plan items with the 'topic:' prefix. Falls back to first 3 plan items if none found.
23–41	for topic in topics: results = run_cypher(...)	For each topic, queries Neo4j for related nodes. Returns topic name, relationship type, related node type, and related node name. Gracefully returns empty list if Neo4j is unavailable.

6.7 Agent Nodes — Node 3: Syllabus

syllabus.py

 File: backend/app/agents/nodes/syllabus.py

 **Purpose:** Cross-references the plan against the course syllabus embeddings. Determines which topics are in-scope vs out-of-scope.

Full Source

```

1  """
2  Node 3: Syllabus – cross-references the plan against the course syllabus embeddings.
3  """
4  import logging
5  from app.agents.state import PipelineState
6  from app.retrieval.hybrid_retriever import hybrid_retriever
7
8  logger = logging.getLogger(__name__)
9
10
11 def syllabus_node(state: PipelineState) -> dict:
12     logger.info("Syllabus node: checking scope")
13     plan = state.get("plan", [])

```

```

14     query = state.get("user_query", "")
15
16     # Retrieve syllabus-tagged documents
17     syllabus_docs = hybrid_retriever.retrieve(
18         f"syllabus scope: {query}",
19         top_k=3,
20         doc_type_filter="syllabus"
21     )
22
23     in_scope = []
24     out_of_scope = []
25
26     if syllabus_docs:
27         syllabus_text = " ".join([d.page_content for d in syllabus_docs])
28         # Simple keyword matching for scope determination
29         for task in plan:
30             task_clean = task.replace("topic:", "").lower()
31             if task_clean in syllabus_text.lower() or len(task_clean) < 5:
32                 in_scope.append(task)
33             else:
34                 # Still include but flag as potentially out of scope
35                 in_scope.append(task)
36     else:
37         # No syllabus uploaded – everything is in scope
38         in_scope = plan
39
40     syllabus_scope = {
41         "in_scope": in_scope,
42         "out_of_scope": out_of_scope,
43         "syllabus_available": len(syllabus_docs) > 0,
44     }
45
46     logger.info(f"Syllabus scope: {len(in_scope)} in, {len(out_of_scope)} out")
47     return {"syllabus_scope": syllabus_scope}
48

```

Line-by-Line Explanation

Lines	Code	What it does
11–21	def syllabus_node(state): syllabus_docs = hybrid_retriever.retrieve(..., doc_type_filter='syllabus')	Retrieves the top-3 syllabus-tagged documents using the hybrid retriever.
23–38	for task in plan: if task_clean in syllabus_text.lower():	Simple keyword matching: checks if each plan item appears in the syllabus text. If no syllabus is uploaded, everything is in-scope.
40–47	return {'syllabus_scope': {...}}	Returns in_scope/out_of_scope lists and a syllabus_available boolean.

6.8 Agent Nodes — Node 4: Notes

notes.py

 File: backend/app/agents/nodes/notes.py

 **Purpose:** Retrieves the most relevant note chunks for the user's question using hybrid (BM25 + FAISS) retrieval with course_id isolation.

Full Source

```
1  """
2  Node 4: Notes – retrieves relevant study notes via vector similarity search (course-
scoped).
3  """
4  import logging
5  from app.agents.state import PipelineState
6  from app.retrieval.hybrid_retriever import hybrid_retriever
7
8  logger = logging.getLogger(__name__)
9
10
11 def notes_node(state: PipelineState) -> dict:
12     logger.info("Notes node: retrieving relevant chunks")
13     query = state.get("user_query", "")
14     plan = state.get("plan", [])
15
16     # Build an enriched search query from plan + original query
17     search_terms = [query] + [p.replace("topic:", "") for p in plan[:3]]
18     enriched_query = " ".join(search_terms)
19
20     docs = hybrid_retriever.retrieve(
21         enriched_query,
22         top_k=5,
23         doc_type_filter="notes",
24         course_id_filter=state.get("course_id")
25     )
26
27     retrieved_notes = []
28     for i, doc in enumerate(docs):
29         retrieved_notes.append({
30             "chunk_index": i,
31             "content": doc.page_content,
32             "source": doc.metadata.get("source", "unknown"),
33             "doc_type": doc.metadata.get("doc_type", "notes"),
34         })
35
36     logger.info(f"Notes retrieved: {len(retrieved_notes)} chunks")
37     return {"retrieved_notes": retrieved_notes}
38
```

Line-by-Line Explanation

Lines	Code	What it does
11–18	search_query = query for item in plan[:3]: search_query += ...	Builds an enriched search query by combining the original question with the

		first 3 plan items to improve retrieval.
20–25	<pre>results = hybrid_retriever.retrieve(search_query, top_k=5, doc_type_filter='notes', course_id_filter=state['course_id'])</pre>	Calls hybrid retrieval with doc_type='notes' and course_id filtering. The course_id_filter prevents cross-course data leakage.
27–36	<pre>retrieved_notes = [{'content': d.page_content, 'source': d.metadata.get('source'), ...}]</pre>	Formats retrieved chunks into a list of dicts with content, source filename, and doc_type for the answer node.

6.8 Agent Nodes — Node 5: PYQ

pyq.py

 File: backend/app/agents/nodes/pyq.py

 **Purpose:** Same as Notes node but filters for doc_type='pyq' (Previous Year Questions). Adds exam-relevance context to the pipeline.

Full Source

```

1  """
2  Node 5: PYQ – retrieves related Previous Year Questions + frequency signal.
3  """
4  import logging
5  from app.agents.state import PipelineState
6  from app.retrieval.hybrid_retriever import hybrid_retriever
7
8  logger = logging.getLogger(__name__)
9
10
11 def pyq_node(state: PipelineState) -> dict:
12     logger.info("PYQ node: retrieving previous year questions")
13     query = state.get("user_query", "")
14     plan = state.get("plan", [])
15
16     search_terms = [query] + [p.replace("topic:", "") for p in plan[:3]]
17     enriched_query = " ".join(search_terms)
18
19     docs = hybrid_retriever.retrieve(
20         enriched_query,
21         top_k=5,
22         doc_type_filter="pyq",
23         course_id_filter=state.get("course_id")
24     )
25
26     related_pyqs = []
27     for i, doc in enumerate(docs):
28         related_pyqs.append({
29             "chunk_index": i,
```

```

30         "content": doc.page_content,
31         "source": doc.metadata.get("source", "unknown"),
32         "year": doc.metadata.get("year", "unknown"),
33     })
34
35     logger.info(f"PYQs retrieved: {len(related_pyqs)} questions")
36     return {"related_pyqs": related_pyqs}
37

```

Line-by-Line Explanation

Lines	Code	What it does
11–23	<pre> results = hybrid_retriever.retrieve(..., doc_type_filter='pyq', course_id_filter=state['course_id']) </pre>	Identical retrieval logic to Notes but filtering for PYQ documents. Uses the same course_id isolation.
25–35	<pre> related_pyqs = [{'content': ..., 'year': ..., 'source': ...}] </pre>	Formats results with year metadata extracted from document metadata.

6.8 Agent Nodes — Node 6: Answer

answer.py

 File: backend/app/agents/nodes/answer.py

 **Purpose:** The LLM synthesizes the final answer from ALL accumulated context — knowledge graph, syllabus scope, notes, PYQs, and any evaluation feedback from a previous retry.

Full Source

```

1  """
2  Node 6: Answer – LLM synthesizes the final answer from all accumulated context with
3  citations.
4  """
5  import logging
6  from app.agents.state import PipelineState
7  from app.agents.llm_setup import get_llm
8
9  logger = logging.getLogger(__name__)
10
11  ANSWER_PROMPT = """You are ExamGPT, an expert AI study assistant. Answer the student's
12  question using ONLY the provided context. Be thorough, accurate, and cite your sources.
13
14  ## Student Question
15  {query}
16
17  ## Knowledge Graph Context
18  {kg_context}
19
20  ## Syllabus Scope
21  {syllabus_scope}
22
23  ## Relevant Notes
24  {notes_context}

```



```

23
24 ## Related Previous Year Questions
25 {pyq_context}
26
27 ## Evaluation Feedback (if retrying)
28 {feedback}
29
30 ## Instructions
31 1. Answer the question thoroughly based on the provided context
32 2. Cite specific sources using [Source: filename] format
33 3. If the question appears out of syllabus scope, mention this
34 4. If related PYQs exist, mention the exam relevance
35 5. Structure your answer with clear headings and bullet points
36
37 ## Answer: ""
38
39
40 def answer_node(state: PipelineState) -> dict:
41     logger.info("Answer node: synthesizing response")
42     llm = get_llm()
43
44     query = state.get("user_query", "")
45     kg_context = state.get("kg_context", [])
46     syllabus_scope = state.get("syllabus_scope", {})
47     retrieved_notes = state.get("retrieved_notes", [])
48     related_pyqs = state.get("related_pyqs", [])
49     evaluation = state.get("evaluation", {})
50
51     # Format contexts for the prompt
52     kg_str = "\n".join([
53         f"- {r.get('topic', '?')} --[{r.get('relationship', '?')}]-->
{r.get('related_name', '?')}"
54         for r in kg_context
55     ]) if kg_context else "No knowledge graph data available."
56
57     syllabus_str = f"In scope: {syllabus_scope.get('in_scope', [])}" if syllabus_scope
else "No syllabus data."
58
59     notes_str = "\n\n".join([
60         f"[Source: {n.get('source', 'unknown')}] \n{n.get('content', '')}"
61         for n in retrieved_notes
62     ]) if retrieved_notes else "No notes available."
63
64     pyq_str = "\n\n".join([
65         f"[Year: {p.get('year', '?')}, Source: {p.get('source',
'unknown')}] \n{p.get('content', '')}"
66         for p in related_pyqs
67     ]) if related_pyqs else "No PYQ data available."
68
69     feedback = evaluation.get("feedback", "None – first attempt.") if evaluation else
"None – first attempt."
70
71     prompt = ANSWER_PROMPT.format(
72         query=query,
73         kg_context=kg_str,
74         syllabus_scope=syllabus_str,
75         notes_context=notes_str,
76         pyq_context=pyq_str,

```

```

77         feedback=feedback,
78     )
79
80     response = llm.invoke(prompt)
81     # ChatHuggingFace returns an AIMessage object; extract string content
82     draft_answer = response.content if hasattr(response, "content") else str(response)
83
84     # Build citations list
85     citations = []
86     for n in retrieved_notes:
87         citations.append({"source": n.get("source"), "type": "notes"})
88     for p in related_pyqs:
89         citations.append({"source": p.get("source"), "type": "pyq", "year":
p.get("year")})
90
91     logger.info(f"Answer generated: {len(draft_answer)} chars, {len(citations)}
citations")
92     return {"draft_answer": draft_answer, "citations": citations}
93

```

Line-by-Line Explanation

Lines	Code	What it does
10–37	ANSWER_PROMPT = """..."""	Detailed prompt with sections for: Student Question, Knowledge Graph Context, Syllabus Scope, Relevant Notes, Related PYQs, and Evaluation Feedback. Instructs the LLM to cite sources using [Source: filename] format.
40–69	kg_str = '\n'.join([...]) notes_str = ... pyq_str = ...	Formats each piece of context into human-readable strings. Uses 'No data available' placeholders for missing context.
71–82	response = llm.invoke(prompt) draft_answer = response.content	Sends the assembled prompt to the LLM and extracts the answer text.
84–92	citations = [] for n in retrieved_notes: citations.append({...})	Builds a citations list from the retrieved notes and PYQs. Each citation includes source filename and type.

6.8 Agent Nodes — Node 7: Evaluation

evaluation.py

 File: backend/app/agents/nodes/evaluation.py

📌 **Purpose:** Critiques the draft answer for correctness, completeness, syllabus alignment, citations, clarity, and relevance. Controls the retry loop.

Full Source

```
1  """
2  Node 7: Evaluation – critiques the draft answer for correctness, syllabus alignment,
completeness.
3  Conditional edge: on fail → loops back to answer_node (bounded ≤3 retries).
4  """
5  import json
6  import logging
7  from app.agents.state import PipelineState
8  from app.agents.llm_setup import get_llm
9  from app.core.config import get_settings
10
11  logger = logging.getLogger(__name__)
12
13  EVAL_PROMPT = """You are a strict academic quality evaluator for ExamGPT. Evaluate the
following AI-generated answer.
14
15  ## Original Question
16  {query}
17
18  ## Draft Answer
19  {draft_answer}
20
21  ## Available Context Used
22  Notes: {has_notes}
23  PYQs: {has_pyqs}
24  Knowledge Graph: {has_kg}
25  Syllabus: {has_syllabus}
26
27  ## Evaluation Criteria
28  1. Correctness: Is the answer factually accurate based on the provided context?
29  2. Completeness: Does it fully address the question?
30  3. Syllabus Alignment: Is the answer within the course scope?
31  4. Citations: Does it reference source material?
32  5. Clarity: Is it well-structured and easy to understand?
33  6. Relevance: Is the answer relevant to the question?
34
35
36  Respond with a JSON object:
37  {{
38    "passed": true/false,
39    "score": 1-10,
40    "feedback": "specific feedback if failed, or 'Approved' if passed"
41  }}
42
43  Respond ONLY with the JSON."""
44
45
46  def evaluation_node(state: PipelineState) -> dict:
47      logger.info("Evaluation node: critiquing answer")
48      llm = get_llm()
49
50      query = state.get("user_query", "")
51      draft_answer = state.get("draft_answer", "")
```

```

52     retry_count = state.get("retry_count", 0)
53
54     prompt = EVAL_PROMPT.format(
55         query=query,
56         draft_answer=draft_answer,
57         has_notes="Yes" if state.get("retrieved_notes") else "No",
58         has_pyqs="Yes" if state.get("related_pyqs") else "No",
59         has_kg="Yes" if state.get("kg_context") else "No",
60         has_syllabus="Yes" if state.get("syllabus_scope",
61     {}).get("syllabus_available") else "No",
62     )
63     response = llm.invoke(prompt)
64     # ChatHuggingFace returns an AIMessage object; extract string content
65     response_text = response.content if hasattr(response, "content") else
str(response)
66
67     try:
68         eval_data = json.loads(response_text.strip())
69         passed = eval_data.get("passed", True)
70         feedback = eval_data.get("feedback", "No feedback")
71         score = eval_data.get("score", 5)
72     except (json.JSONDecodeError, AttributeError):
73         # If LLM can't produce valid JSON, pass the answer through
74         passed = True
75         feedback = "Evaluation parsing failed – passing through."
76         score = 5
77
78     evaluation = {
79         "passed": passed,
80         "score": score,
81         "feedback": feedback,
82     }
83
84     logger.info(f"Evaluation: passed={passed}, score={score}, retry={retry_count}")
85     return {
86         "evaluation": evaluation,
87         "retry_count": retry_count + (0 if passed else 1),
88     }
89
90
91 def should_retry(state: PipelineState) -> str:
92     """Conditional edge: retry answer if evaluation failed and under retry limit."""
93     settings = get_settings()
94     evaluation = state.get("evaluation", {})
95     retry_count = state.get("retry_count", 0)
96
97     if not evaluation.get("passed", True) and retry_count < settings.MAX_EVAL_RETRIES:
98         logger.info(f"Retrying answer (attempt {retry_count +
99     1}/{settings.MAX_EVAL_RETRIES}")
100         return "retry"
101     return "end"
102

```

Line-by-Line Explanation

Lines	Code	What it does
-------	------	--------------

13–43	EVAL_PROMPT = """..."""	Prompt listing 6 evaluation criteria. Instructs the LLM to return JSON: {passed: bool, score: 1-10, feedback: string}.
46–88	def evaluation_node(state):	Sends the evaluation prompt. Parses the JSON response. If parsing fails, passes the answer through (assumes adequate). Increments retry_count on failure.
91–101	def should_retry(state):	Conditional edge function. Returns 'retry' if evaluation failed AND retry_count < MAX_EVAL_RETRIES (3). Otherwise returns 'end'.

6.9 Document Ingestion — PDF Loader

pdf_loader.py

 File: backend/app/ingestion/pdf_loader.py

 **Purpose:** Extracts text from PDF files page by page using pypdf.

Full Source

```

1  import pypdf
2
3  class PDFLoader:
4      @staticmethod
5      def extract_text(file_path: str) -> str:
6          text = ""
7          try:
8              with open(file_path, "rb") as f:
9                  reader = pypdf.PdfReader(f)
10                 for page in reader.pages:
11                     extracted = page.extract_text()
12                     if extracted:
13                         text += extracted + "\n"
14             except Exception as e:
15                 print(f"Error extracting PDF text: {e}")
16             return text
17

```

Line-by-Line Explanation

Lines	Code	What it does
1–16	class PDFLoader: @staticmethod def extract_text(file_path):	Opens the PDF in binary mode, creates a PdfReader, loops through pages calling extract_text(). Concatenates all text with newlines. Silently

		skips pages that fail.
--	--	------------------------

6.9 Document Ingestion — Text Chunker

chunker.py

 File: backend/app/ingestion/chunker.py

 **Purpose:** Splits extracted text into overlapping chunks suitable for embedding and retrieval.

Full Source

```
1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2 from langchain_core.documents import Document
3
4 class Chunker:
5     def __init__(self, chunk_size: int = 1000, chunk_overlap: int = 200):
6         self.splitter = RecursiveCharacterTextSplitter(
7             chunk_size=chunk_size,
8             chunk_overlap=chunk_overlap,
9             separators=["\n\n", "\n", " ", ""]
10        )
11
12    def chunk_text(self, text: str, metadata: dict = None) -> list[Document]:
13        if metadata is None:
14            metadata = {}
15        return self.splitter.create_documents([text], metadatas=[metadata])
16
```

Line-by-Line Explanation

Lines	Code	What it does
4–10	class Chunker: def __init__(self, chunk_size=1000, chunk_overlap=200):	chunk_size=1000 chars per chunk, chunk_overlap=200 chars shared between consecutive chunks. Overlap ensures concepts spanning chunk boundaries aren't lost. Split hierarchy: paragraphs → newlines → spaces → chars.
12–15	def chunk_text(self, text, metadata):	Returns a list of LangChain Document objects, each with page_content (chunk text) and metadata (source, course_id, doc_type).

6.9 Document Ingestion — Embedder

embedder.py

 File: backend/app/ingestion/embedder.py

📌 **Purpose:** Wraps the HuggingFace embedding model to convert text into 384-dimensional vectors.

Full Source

```
1 from app.agents.llm_setup import get_embeddings
2
3
4 class Embedder:
5     def __init__(self):
6         self.model = get_embeddings()
7
8     def embed_documents(self, texts: list[str]) -> list[list[float]]:
9         return self.model.embed_documents(texts)
10
11     def embed_query(self, text: str) -> list[float]:
12         return self.model.embed_query(text)
13
```

Line-by-Line Explanation

Lines	Code	What it does
4–12	<pre>class Embedder: def embed_documents(self, texts): def embed_query(self, text):</pre>	embed_documents: batch-converts text list to vectors. embed_query: converts a single query to a vector. Both call the HF Inference API.

6.9 Document Ingestion — FAISS Vector Store

vector_store.py

📄 File: backend/app/ingestion/vector_store.py

📌 **Purpose:** Manages the FAISS index: creating, loading, saving, adding embeddings, and searching.

Full Source

```
1 import os
2 import faiss
3 import numpy as np
4
5 VECTOR_STORE_DIR = os.getenv("VECTOR_STORE_DIR", "/app/vector_store")
6
7 class VectorStore:
8     def __init__(self, index_name="examgpt_index"):
9         self.index_name = index_name
10        self.index_path = os.path.join(VECTOR_STORE_DIR, f"{index_name}.index")
11        self.index = None
12        self.dimension = 384
13        self._load_index()
14
15    def _load_index(self):
```

```

16         if os.path.exists(self.index_path):
17             self.index = faiss.read_index(self.index_path)
18         else:
19             self.index = faiss.IndexFlatL2(self.dimension)
20
21     def save_index(self):
22         os.makedirs(VECTOR_STORE_DIR, exist_ok=True)
23         faiss.write_index(self.index, self.index_path)
24
25     def add_embeddings(self, embeddings: list[list[float]]):
26         if not embeddings:
27             return
28         embeddings_np = np.array(embeddings).astype('float32')
29         self.index.add(embeddings_np)
30         self.save_index()
31
32     def search(self, query_embedding: list[float], top_k: int = 4):
33         query_np = np.array([query_embedding]).astype('float32')
34         distances, indices = self.index.search(query_np, top_k)
35         return distances, indices
36

```

Line-by-Line Explanation

Lines	Code	What it does
7–13	<pre> class VectorStore: def __init__(self, index_name): self.dimension = 384 </pre>	dimension=384 matches the MiniLM model output. Tries to load an existing index from disk; creates a new IndexFlatL2 if none found.
21–23	<pre> def save_index(self): </pre>	Writes the FAISS index to disk for persistence across restarts.
25–35	<pre> def add_embeddings(self, embeddings): def search(self, query_embedding, top_k): </pre>	add_embeddings: converts to float32 numpy array, adds to index, saves. search: returns top_k nearest neighbors by L2 distance.

6.10 Retrieval System — FAISS Semantic Retriever

faiss_retriever.py

 File: backend/app/retrieval/faiss_retriever.py

 **Purpose:** Wraps VectorStore and Embedder for semantic search: converts query to vector, finds nearest neighbors.

Full Source

```

1 from app.ingestion.vector_store import VectorStore
2 from app.ingestion.embedder import Embedder
3

```



```

4 class FAISSRetriever:
5     def __init__(self):
6         self.vector_store = VectorStore()
7         self.embedder = Embedder()
8
9     def search(self, query: str, top_k: int = 4):
10        query_embedding = self.embedder.embed_query(query)
11        distances, indices = self.vector_store.search(query_embedding, top_k)
12        return indices[0], distances[0]
13

```

Line-by-Line Explanation

Lines	Code	What it does
4–12	<pre> class FAISSRetriever: def search(self, query, top_k): </pre>	Embeds the query text into a 384-dim vector, then searches the FAISS index for the top_k most similar document vectors by Euclidean distance.

6.10 Retrieval System — BM25 Keyword Retriever

bm25_retriever.py

 File: backend/app/retrieval/bm25_retriever.py

 **Purpose:** Traditional keyword matching using the BM25 (Okapi BM25) algorithm. Considers term frequency and inverse document frequency.

Full Source

```

1 from rank_bm25 import BM25Okapi
2
3 class BM25Retriever:
4     def __init__(self):
5         self.bm25 = None
6         self.documents = []
7
8     def build_index(self, texts: list[str], documents: list):
9         if not texts:
10            return
11            self.documents = documents
12            tokenized_corpus = [doc.split(" ") for doc in texts]
13            self.bm25 = BM25Okapi(tokenized_corpus)
14
15    def search(self, query: str, top_k: int = 4):
16        if not self.bm25:
17            return [], []
18            tokenized_query = query.split(" ")
19            scores = self.bm25.get_scores(tokenized_query)
20            import numpy as np
21            top_indices = np.argsort(scores)[::-1][:top_k]
22            return top_indices, [scores[i] for i in top_indices]
23

```

Line-by-Line Explanation

Lines	Code	What it does
3–13	<pre>class BM25Retriever: def build_index(self, texts, documents):</pre>	Tokenizes each document text by splitting on spaces, then builds a BM25 index. Stores the original Document objects for later retrieval.
15–22	<pre>def search(self, query, top_k):</pre>	Tokenizes the query, scores all documents, sorts by score (highest first), returns top_k indices and scores.

6.10 Retrieval System — Hybrid Retriever (Semantic + Keyword)

hybrid_retriever.py

 File: backend/app/retrieval/hybrid_retriever.py

 **Purpose:** Combines BM25 and FAISS results using Reciprocal Rank Fusion (RRF). Filters by doc_type and course_id for isolation.

Full Source

```
1 from app.retrieval.bm25_retriever import BM25Retriever
2 from app.retrieval.faiss_retriever import FAISSRetriever
3 from langchain_core.documents import Document
4
5
6 class HybridRetriever:
7     def __init__(self):
8         self.bm25_retriever = BM25Retriever()
9         self.faiss_retriever = FAISSRetriever()
10        self.documents = []
11
12    def add_documents(self, docs: list[Document]):
13        if not docs:
14            return
15        self.documents.extend(docs)
16        texts = [doc.page_content for doc in docs]
17
18        # Rebuild BM25
19        self.bm25_retriever.build_index(
20            [d.page_content for d in self.documents], self.documents
21        )
22
23        # Build FAISS
24        embeddings = self.faiss_retriever.embedder.embed_documents(texts)
25        self.faiss_retriever.vector_store.add_embeddings(embeddings)
26
27    def retrieve(self, query: str, top_k: int = 4, doc_type_filter: str = None,
course_id_filter: int = None) -> list[Document]:
28        """Retrieve documents with optional doc_type and course_id filtering."""
29        if not self.documents:
```

```

30         return []
31
32     # Get enough candidates to survive filtering
33     search_k = top_k * 10
34     bm25_indices, _ = self.bm25_retriever.search(query, search_k)
35     faiss_indices, _ = self.faiss_retriever.search(query, search_k)
36
37     fused_scores = {}
38     for rank, doc_idx in enumerate(bm25_indices):
39         fused_scores[doc_idx] = fused_scores.get(doc_idx, 0) + 1 / (rank + 60)
40
41     for rank, doc_idx in enumerate(faiss_indices):
42         if doc_idx != -1 and doc_idx < len(self.documents):
43             fused_scores[doc_idx] = fused_scores.get(doc_idx, 0) + 1 / (rank + 60)
44
45     sorted_indices = sorted(fused_scores.keys(), key=lambda x: fused_scores[x],
reverse=True)
46
47     results = []
48     for idx in sorted_indices:
49         if idx < len(self.documents):
50             doc = self.documents[idx]
51
52             # Apply filters
53             if doc_type_filter and doc.metadata.get("doc_type", "notes") !=
doc_type_filter:
54                 continue
55             if course_id_filter is not None and doc.metadata.get("course_id") !=
course_id_filter:
56                 continue
57
58             results.append(doc)
59             if len(results) >= top_k:
60                 break
61
62     return results
63
64
65 # Global instance
66 hybrid_retriever = HybridRetriever()
67

```

Line-by-Line Explanation

Lines	Code	What it does
6–10	class HybridRetriever: def __init__(self):	Holds both retrievers and maintains a master list of all indexed documents.
12–25	def add_documents(self, docs):	Adds new documents to both indexes simultaneously: extends the master list, rebuilds BM25 with ALL documents, adds new embeddings to FAISS.
27–62	def retrieve(self, query, top_k, doc_type_filter,	Core hybrid search. Gets 10x candidates from both BM25

	course_id_filter):	and FAISS. Fuses with RRF: score = $\Sigma(1/(\text{rank} + 60))$ across both result lists. Filters by doc_type and course_id. Returns top_k.
--	--------------------	---

6.11 API Endpoints — Authentication

auth.py

 File: backend/app/api/auth.py

 **Purpose:** Register, login, refresh, logout, and /me endpoints for user authentication.

Full Source

```

1  """
2  Auth API routes – register, login, refresh, logout, me.
3  """
4  from fastapi import APIRouter, Depends, HTTPException, status
5  from sqlalchemy.orm import Session
6  from fastapi.security import OAuth2PasswordRequestForm
7  from app.db.database import get_db
8  from app.db.models import User
9  from app.schemas.auth import UserCreate, UserResponse, Token
10 from app.core.security import (
11     get_password_hash, verify_password, create_access_token,
12     create_refresh_token, get_current_user,
13 )
14 from app.core.config import get_settings
15
16 router = APIRouter()
17
18
19 @router.post("/register", response_model=UserResponse,
status_code=status.HTTP_201_CREATED)
20 def register(user_in: UserCreate, db: Session = Depends(get_db)):
21     # Check existing
22     if db.query(User).filter(User.username == user_in.username).first():
23         raise HTTPException(status_code=400, detail="Username already registered")
24     if db.query(User).filter(User.email == user_in.email).first():
25         raise HTTPException(status_code=400, detail="Email already registered")
26
27     user = User(
28         username=user_in.username,
29         email=user_in.email,
30         name=user_in.name,
31         password_hash=get_password_hash(user_in.password),
32         role="student",
33     )
34     db.add(user)
35     db.commit()
36     db.refresh(user)
37     return user
38

```

```

39
40 @router.post("/login", response_model=Token)
41 def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session =
Depends(get_db)):
42     user = db.query(User).filter(User.username == form_data.username).first()
43     if not user or not verify_password(form_data.password, user.password_hash):
44         raise HTTPException(
45             status_code=status.HTTP_401_UNAUTHORIZED,
46             detail="Incorrect username or password",
47             headers={"WWW-Authenticate": "Bearer"},
48         )
49
50     access_token = create_access_token(subject=user.username)
51     return Token(access_token=access_token, token_type="bearer")
52
53
54 @router.post("/refresh", response_model=Token)
55 def refresh_token(db: Session = Depends(get_db), current_user: User =
Depends(get_current_user)):
56     """Issue a new access token for an authenticated user."""
57     access_token = create_access_token(subject=current_user.username)
58     return Token(access_token=access_token, token_type="bearer")
59
60
61 @router.post("/logout")
62 def logout(current_user: User = Depends(get_current_user)):
63     """Logout – in production, revoke refresh token from Redis."""
64     return {"message": "Logged out successfully"}
65
66
67 @router.get("/me", response_model=UserResponse)
68 def read_users_me(current_user: User = Depends(get_current_user)):
69     return current_user
70

```

Line-by-Line Explanation

Lines	Code	What it does
19–37	POST /register	Creates a new user. Checks username/email uniqueness. Hashes password with bcrypt. Creates User record in PostgreSQL.
40–51	POST /login	Authenticates via OAuth2PasswordRequestForm (form-encoded). Verifies password against stored hash. Returns JWT access token.
54–64	POST /refresh, POST /logout	refresh: issues a new access token for already-authenticated users. logout: returns success (production would revoke tokens from Redis).

67–69	GET /me	Returns the currently authenticated user's profile using the <code>get_current_user</code> dependency.
-------	---------	--

6.11 API Endpoints — Courses

`courses.py`

 File: `backend/app/api/courses.py`

 **Purpose:** CRUD operations for course management. All endpoints are owner-scoped.

Full Source

```

1  """
2  Course API routes – CRUD operations for course management.
3  All endpoints are course-scoped and owner-authenticated.
4  """
5  from fastapi import APIRouter, Depends, HTTPException, status
6  from sqlalchemy.orm import Session
7  from app.db.database import get_db
8  from app.db.models import Course, User, Document
9  from app.schemas.course import CourseCreate, CourseResponse, CourseListResponse
10 from app.core.security import get_current_user
11
12 router = APIRouter()
13
14
15 @router.post("/", response_model=CourseResponse, status_code=status.HTTP_201_CREATED)
16 def create_course(
17     course_in: CourseCreate,
18     db: Session = Depends(get_db),
19     current_user: User = Depends(get_current_user),
20 ):
21     course = Course(
22         name=course_in.name,
23         description=course_in.description,
24         user_id=current_user.id,
25     )
26     db.add(course)
27     db.commit()
28     db.refresh(course)
29     return CourseResponse(
30         id=course.id,
31         name=course.name,
32         description=course.description,
33         user_id=course.user_id,
34         created_at=course.created_at,
35         document_count=0,
36     )
37
38
39 @router.get("/", response_model=CourseListResponse)
40 def list_courses(
41     db: Session = Depends(get_db),

```

```

42     current_user: User = Depends(get_current_user),
43 ):
44     courses = db.query(Course).filter(Course.user_id == current_user.id).all()
45     course_responses = []
46     for c in courses:
47         doc_count = db.query(Document).filter(Document.course_id == c.id).count()
48         course_responses.append(CourseResponse(
49             id=c.id,
50             name=c.name,
51             description=c.description,
52             user_id=c.user_id,
53             created_at=c.created_at,
54             document_count=doc_count,
55         ))
56     return CourseListResponse(courses=course_responses, total=len(course_responses))
57
58
59 @router.get("/{course_id}", response_model=CourseResponse)
60 def get_course(
61     course_id: int,
62     db: Session = Depends(get_db),
63     current_user: User = Depends(get_current_user),
64 ):
65     course = db.query(Course).filter(
66         Course.id == course_id, Course.user_id == current_user.id
67     ).first()
68     if not course:
69         raise HTTPException(status_code=404, detail="Course not found")
70     doc_count = db.query(Document).filter(Document.course_id == course.id).count()
71     return CourseResponse(
72         id=course.id,
73         name=course.name,
74         description=course.description,
75         user_id=course.user_id,
76         created_at=course.created_at,
77         document_count=doc_count,
78     )
79
80
81 @router.delete("/{course_id}", status_code=status.HTTP_204_NO_CONTENT)
82 def delete_course(
83     course_id: int,
84     db: Session = Depends(get_db),
85     current_user: User = Depends(get_current_user),
86 ):
87     course = db.query(Course).filter(
88         Course.id == course_id, Course.user_id == current_user.id
89     ).first()
90     if not course:
91         raise HTTPException(status_code=404, detail="Course not found")
92     db.delete(course)
93     db.commit()
94

```

Line-by-Line Explanation

Lines	Code	What it does
15–36	POST /	Creates a new course linked

		to the authenticated user.
39–56	GET /	Lists all courses for the authenticated user with document counts.
59–78	GET /{course_id}	Gets a single course by ID. Double-filter (course_id + user_id) ensures owner-only access.
81–93	DELETE /{course_id}	Deletes a course after verifying ownership.

6.11 API Endpoints — Documents (Upload & Ingestion)

documents.py

 File: backend/app/api/documents.py

 **Purpose:** Upload PDFs, validate them, extract text, chunk, embed, and index into the hybrid retriever.

Full Source

```

1  """
2  Document API routes – upload and list documents for a course.
3  """
4  import os
5  from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, Form, status
6  from sqlalchemy.orm import Session
7  from app.db.database import get_db
8  from app.db.models import Course, Document, User, DocumentStatus, DocumentType
9  from app.schemas.document import DocumentUploadResponse, DocumentResponse,
DocumentListResponse
10 from app.core.security import get_current_user
11 from app.core.config import get_settings
12 from app.ingestion.pdf_loader import PDFLoader
13 from app.ingestion.chunker import Chunker
14 from app.retrieval.hybrid_retriever import hybrid_retriever
15 from langchain_core.documents import Document as LCDocument
16
17 router = APIRouter()
18
19
20 def _verify_course_owner(db: Session, course_id: int, user_id: int) -> Course:
21     course = db.query(Course).filter(
22         Course.id == course_id, Course.user_id == user_id
23     ).first()
24     if not course:
25         raise HTTPException(status_code=404, detail="Course not found")
26     return course
27
28
29 @router.post("/{course_id}/documents", response_model=DocumentUploadResponse)
30 def upload_document(
31     course_id: int,
```



```

32     file: UploadFile = File(...),
33     doc_type: str = Form(default="notes"),
34     db: Session = Depends(get_db),
35     current_user: User = Depends(get_current_user),
36 ):
37     settings = get_settings()
38     course = _verify_course_owner(db, course_id, current_user.id)
39
40     # Validate file type
41     if not file.filename.lower().endswith(".pdf"):
42         raise HTTPException(status_code=400, detail="Only PDF files are supported")
43
44     # Validate file size
45     file_data = file.file.read()
46     if len(file_data) > settings.MAX_UPLOAD_SIZE_MB * 1024 * 1024:
47         raise HTTPException(status_code=400, detail=f"File exceeds
{settings.MAX_UPLOAD_SIZE_MB}MB limit")
48
49     # Validate doc_type
50     if doc_type not in [dt.value for dt in DocumentType]:
51         doc_type = DocumentType.notes.value
52
53     # Save to DB with processing status
54     db_doc = Document(
55         title=file.filename,
56         filename=file.filename,
57         doc_type=doc_type,
58         status=DocumentStatus.processing.value,
59         course_id=course.id,
60         user_id=current_user.id,
61     )
62     db.add(db_doc)
63     db.commit()
64     db.refresh(db_doc)
65
66     # Process synchronously (Celery can be added later)
67     try:
68         # Save file locally for processing
69         upload_dir = os.path.join(settings.UPLOAD_DIR, str(course_id))
70         os.makedirs(upload_dir, exist_ok=True)
71         file_path = os.path.join(upload_dir, file.filename)
72         with open(file_path, "wb") as f:
73             f.write(file_data)
74
75         # Try Supabase upload (graceful fallback if not available)
76         try:
77             from app.core.supabase_client import upload_file
78             object_name = f"courses/{course_id}/{file.filename}"
79             db_doc.file_url = upload_file(file_data, object_name)
80         except Exception:
81             db_doc.file_url = file_path
82
83         # Extract text
84         text = PDFLoader.extract_text(file_path)
85
86         # Chunk and index
87         chunker = Chunker()
88         docs = chunker.chunk_text(text, metadata={

```

```

89         "source": file.filename,
90         "user_id": current_user.id,
91         "course_id": course_id,
92         "doc_type": doc_type,
93     })
94     hybrid_retriever.add_documents(docs)
95
96     # Update status
97     db_doc.status = DocumentStatus.ready.value
98     db.commit()
99
100 except Exception as e:
101     db_doc.status = DocumentStatus.failed.value
102     db.commit()
103     import traceback
104     traceback.print_exc()
105     raise HTTPException(status_code=500, detail=f"Processing failed: {repr(e)}")
106
107 return DocumentUploadResponse(
108     id=db_doc.id,
109     filename=db_doc.filename,
110     doc_type=db_doc.doc_type,
111     status=db_doc.status,
112     message="Upload successful and indexed.",
113 )
114
115
116 @router.get("/{course_id}/documents", response_model=DocumentListResponse)
117 def list_documents(
118     course_id: int,
119     db: Session = Depends(get_db),
120     current_user: User = Depends(get_current_user),
121 ):
122     _verify_course_owner(db, course_id, current_user.id)
123     docs = db.query(Document).filter(Document.course_id == course_id).all()
124     doc_responses = [
125         DocumentResponse(
126             id=d.id,
127             title=d.title,
128             filename=d.filename,
129             doc_type=d.doc_type,
130             status=d.status,
131             course_id=d.course_id,
132             created_at=d.created_at,
133         )
134         for d in docs
135     ]
136     return DocumentListResponse(documents=doc_responses, total=len(doc_responses))
137

```

Line-by-Line Explanation

Lines	Code	What it does
20–26	def _verify_course_owner(db, course_id, user_id):	Helper function that checks course ownership. Used by both upload and list endpoints.

29–105	POST /{course_id}/documents	The full upload pipeline: (1) validate .pdf + size, (2) create DB record with status=processing, (3) save locally, (4) try Supabase upload, (5) extract text with PDFLoader, (6) chunk with Chunker, (7) index into hybrid_retriever, (8) update status to ready. On failure: status=failed.
116–136	GET /{course_id}/documents	Lists all documents for a course with filename, type, status, and upload date.

6.11 API Endpoints — Query (AI Chat)

query.py

 File: backend/app/api/query.py

 **Purpose:** The main chat endpoint. Triggers the full 7-node LangGraph pipeline and saves results.

Full Source

```

1  """
2  Query API routes – submit questions and retrieve chat history.
3  Triggers the full 7-node LangGraph pipeline.
4  """
5  from fastapi import APIRouter, Depends, HTTPException
6  from sqlalchemy.orm import Session
7  from app.db.database import get_db
8  from app.db.models import Course, ChatHistory, PipelineRun, User
9  from app.schemas.query import ChatRequest, ChatResponse, ChatHistoryItem,
ChatHistoryResponse
10 from app.core.security import get_current_user
11 from app.agents.graph import run_examgpt_query
12
13 router = APIRouter()
14
15
16 @router.post("/{course_id}/query", response_model=ChatResponse)
17 def submit_query(
18     course_id: int,
19     request: ChatRequest,
20     db: Session = Depends(get_db),
21     current_user: User = Depends(get_current_user),
22 ):
23     # Verify course ownership
24     course = db.query(Course).filter(
25         Course.id == course_id, Course.user_id == current_user.id
26     ).first()

```

```

27     if not course:
28         raise HTTPException(status_code=404, detail="Course not found")
29
30     # Run the full pipeline
31     result = run_examgpt_query(
32         query=request.query,
33         user_id=current_user.id,
34         course_id=course_id,
35     )
36
37     # Save chat history
38     chat = ChatHistory(
39         user_id=current_user.id,
40         course_id=course_id,
41         query=request.query,
42         response=result["answer"],
43         citations_json=result.get("citations", []),
44     )
45     db.add(chat)
46     db.commit()
47     db.refresh(chat)
48
49     # Save pipeline run for observability
50     pipeline_run = PipelineRun(
51         chat_id=chat.id,
52         course_id=course_id,
53         user_id=current_user.id,
54         retry_count=result.get("retry_count", 0),
55         total_latency_ms=result.get("latency_ms", 0),
56         status="completed" if not result.get("error") else "failed",
57     )
58     db.add(pipeline_run)
59     db.commit()
60
61     return ChatResponse(
62         response=result["answer"],
63         citations=result.get("citations", []),
64         evaluation=result.get("evaluation", {}),
65         retry_count=result.get("retry_count", 0),
66         latency_ms=result.get("latency_ms", 0),
67     )
68
69
70 @router.get("/{course_id}/chat-history", response_model=ChatHistoryResponse)
71 def get_chat_history(
72     course_id: int,
73     db: Session = Depends(get_db),
74     current_user: User = Depends(get_current_user),
75 ):
76     course = db.query(Course).filter(
77         Course.id == course_id, Course.user_id == current_user.id
78     ).first()
79     if not course:
80         raise HTTPException(status_code=404, detail="Course not found")
81
82     chats = db.query(ChatHistory).filter(
83         ChatHistory.course_id == course_id,
84         ChatHistory.user_id == current_user.id,

```

```

85     ).order_by(ChatHistory.created_at.desc()).limit(50).all()
86
87     messages = [
88         ChatHistoryItem(
89             id=c.id,
90             query=c.query,
91             response=c.response,
92             citations_json=c.citations_json,
93             created_at=c.created_at,
94         )
95         for c in chats
96     ]
97     return ChatHistoryResponse(messages=messages, total=len(messages))
98

```

Line-by-Line Explanation

Lines	Code	What it does
16–67	POST <code>/[course_id]/query</code>	The chat endpoint: (1) verify course ownership, (2) call <code>run_examgpt_query()</code> (runs full 7-node pipeline), (3) save Q&A to ChatHistory, (4) create PipelineRun record for observability, (5) return answer + citations + evaluation + latency.
70–97	GET <code>/[course_id]/chat-history</code>	Retrieves the last 50 chat messages for a course, ordered newest first.

6.11 API Endpoints — Study Plan

study_plan.py

 File: `backend/app/api/study_plan.py`

 **Purpose:** Generates AI study plans using the LLM directly (not through the pipeline).

Full Source

```

1  """
2  Study Plan API routes – generate and retrieve study plans for a course.
3  """
4  import json
5  import logging
6  from fastapi import APIRouter, Depends, HTTPException
7  from sqlalchemy.orm import Session
8  from app.db.database import get_db
9  from app.db.models import Course, StudyPlan, Document, User
10 from app.schemas.planner import StudyPlanRequest, StudyPlanResponse
11 from app.core.security import get_current_user
12 from app.agents.llm_setup import get_llm
13

```

```

14 logger = logging.getLogger(__name__)
15 router = APIRouter()
16
17 STUDY_PLAN_PROMPT = """You are ExamGPT, a study planning expert. Generate a structured
study plan based on the following course information.
18
19 Course: {course_name}
20 Available days: {days}
21 Hours per day: {hours_per_day}
22 Total available hours: {total_hours}
23
24 Document types available:
25 - Syllabus documents: {syllabus_count}
26 - Notes documents: {notes_count}
27 - PYQ documents: {pyq_count}
28
29 Generate a study plan as a JSON object with this structure:
30 {{
31     "title": "Study Plan for [course name]",
32     "total_days": {days},
33     "hours_per_day": {hours_per_day},
34     "phases": [
35         {{
36             "phase": 1,
37             "name": "Phase name",
38             "days": "Day X - Day Y",
39             "focus": "What to focus on",
40             "topics": ["topic1", "topic2"],
41             "activities": ["Read notes on X", "Practice PYQs for Y"],
42             "hours": 10
43         }}
44     ],
45     "tips": ["Study tip 1", "Study tip 2"],
46     "priority_topics": ["high priority topic 1", "topic 2"]
47 }}
48
49 Respond ONLY with the JSON."""
50
51
52 @router.post("/{course_id}/study-plan", response_model=StudyPlanResponse)
53 def generate_study_plan(
54     course_id: int,
55     request: StudyPlanRequest,
56     db: Session = Depends(get_db),
57     current_user: User = Depends(get_current_user),
58 ):
59     course = db.query(Course).filter(
60         Course.id == course_id, Course.user_id == current_user.id
61     ).first()
62     if not course:
63         raise HTTPException(status_code=404, detail="Course not found")
64
65     # Count documents by type
66     syllabus_count = db.query(Document).filter(
67         Document.course_id == course_id, Document.doc_type == "syllabus"
68     ).count()
69     notes_count = db.query(Document).filter(
70         Document.course_id == course_id, Document.doc_type == "notes"

```

```

71     ).count()
72     pyq_count = db.query(Document).filter(
73         Document.course_id == course_id, Document.doc_type == "pyq"
74     ).count()
75
76     llm = get_llm()
77     total_hours = request.days * request.hours_per_day
78
79     prompt = STUDY_PLAN_PROMPT.format(
80         course_name=course.name,
81         days=request.days,
82         hours_per_day=request.hours_per_day,
83         total_hours=total_hours,
84         syllabus_count=syllabus_count,
85         notes_count=notes_count,
86         pyq_count=pyq_count,
87     )
88
89     response = llm.invoke(prompt)
90
91     try:
92         plan_data = json.loads(response.strip())
93     except (json.JSONDecodeError, AttributeError):
94         plan_data = {
95             "title": f"Study Plan for {course.name}",
96             "total_days": request.days,
97             "hours_per_day": request.hours_per_day,
98             "phases": [{"phase": 1, "name": "General Study", "days": f"Day 1 - Day
{request.days}", "focus": "Review all materials", "topics": [], "activities": ["Review
notes", "Practice PYQs"], "hours": total_hours}],
99             "tips": ["Start with syllabus topics", "Practice PYQs regularly"],
100             "priority_topics": [],
101         }
102
103     # Save or update
104     existing = db.query(StudyPlan).filter(
105         StudyPlan.course_id == course_id, StudyPlan.user_id == current_user.id
106     ).first()
107
108     if existing:
109         existing.plan_data = plan_data
110         db.commit()
111         db.refresh(existing)
112         plan = existing
113     else:
114         plan = StudyPlan(
115             course_id=course_id,
116             user_id=current_user.id,
117             plan_data=plan_data,
118         )
119         db.add(plan)
120         db.commit()
121         db.refresh(plan)
122
123     return StudyPlanResponse(
124         id=plan.id,
125         course_id=plan.course_id,
126         plan_data=plan.plan_data,

```

```

127         created_at=plan.created_at,
128     )
129
130
131 @router.get("/{course_id}/study-plan", response_model=StudyPlanResponse)
132 def get_study_plan(
133     course_id: int,
134     db: Session = Depends(get_db),
135     current_user: User = Depends(get_current_user),
136 ):
137     course = db.query(Course).filter(
138         Course.id == course_id, Course.user_id == current_user.id
139     ).first()
140     if not course:
141         raise HTTPException(status_code=404, detail="Course not found")
142
143     plan = db.query(StudyPlan).filter(
144         StudyPlan.course_id == course_id, StudyPlan.user_id == current_user.id
145     ).order_by(StudyPlan.created_at.desc()).first()
146
147     if not plan:
148         raise HTTPException(status_code=404, detail="No study plan found. Generate one
first.")
149
150     return StudyPlanResponse(
151         id=plan.id,
152         course_id=plan.course_id,
153         plan_data=plan.plan_data,
154         created_at=plan.created_at,
155     )
156

```

Line-by-Line Explanation

Lines	Code	What it does
17–49	STUDY_PLAN_PROMPT	Detailed prompt template instructing the LLM to generate a structured JSON study plan with phases, topics, activities, and tips.
52–128	POST /{course_id}/study-plan	Counts documents by type, formats the prompt, sends to LLM, parses JSON response (with fallback default). Either updates existing plan or creates new one.
131–155	GET /{course_id}/study-plan	Retrieves the most recent study plan for a course. Returns 404 if none exists.

6.12 Pydantic Schemas — Auth

auth.py

 File: backend/app/schemas/auth.py

 **Purpose:** Request/response models for authentication endpoints.

Full Source

```
1  from pydantic import BaseModel, EmailStr
2  from typing import Optional
3
4
5  class UserCreate(BaseModel):
6      username: str
7      email: str
8      name: str
9      password: str
10
11
12  class UserResponse(BaseModel):
13      id: int
14      username: str
15      email: str
16      name: str
17      role: str = "student"
18
19      class Config:
20          from_attributes = True
21
22
23  class Token(BaseModel):
24      access_token: str
25      token_type: str = "bearer"
26
27
28  class TokenRefresh(BaseModel):
29      access_token: str
30      token_type: str = "bearer"
31
32
33  class LoginRequest(BaseModel):
34      username: str
35      password: str
36
```

Line-by-Line Explanation

Lines	Code	What it does
5–9	class UserCreate(BaseModel):	Registration input: username, email, name, password. All required strings.
12–20	class UserResponse(BaseModel):	Registration/me output. Note: password_hash is NOT included.

		from_attributes=True enables conversion from SQLAlchemy objects.
23–35	class Token / TokenRefresh / LoginRequest	Token: contains access_token + type. Used as the login response model.

6.12 Pydantic Schemas — Course

course.py

 File: backend/app/schemas/course.py

 **Purpose:** Request/response models for course CRUD.

Full Source

```

1  from pydantic import BaseModel
2  from typing import Optional
3  from datetime import datetime
4
5
6  class CourseCreate(BaseModel):
7      name: str
8      description: Optional[str] = None
9
10
11  class CourseResponse(BaseModel):
12      id: int
13      name: str
14      description: Optional[str] = None
15      user_id: int
16      created_at: datetime
17      document_count: int = 0
18
19      class Config:
20          from_attributes = True
21
22
23  class CourseListResponse(BaseModel):
24      courses: list[CourseResponse]
25      total: int
26

```

Line-by-Line Explanation

Lines	Code	What it does
6–25	CourseCreate / CourseResponse / CourseListResponse	CourseCreate: just name + optional description. CourseResponse adds id, user_id, created_at, document_count. CourseListResponse wraps a list + total count.

6.12 Pydantic Schemas — Document

document.py

File: backend/app/schemas/document.py

Purpose: Models for document upload responses and document listings.

Full Source

```
1 from pydantic import BaseModel
2 from typing import Optional
3 from datetime import datetime
4
5
6 class DocumentUploadResponse(BaseModel):
7     id: int
8     filename: str
9     doc_type: str
10    status: str
11    message: str
12
13    class Config:
14        from_attributes = True
15
16
17 class DocumentResponse(BaseModel):
18     id: int
19     title: str
20     filename: str
21     doc_type: str
22     status: str
23     course_id: int
24     created_at: datetime
25
26     class Config:
27         from_attributes = True
28
29
30 class DocumentListResponse(BaseModel):
31     documents: list[DocumentResponse]
32     total: int
33
```

Line-by-Line Explanation

Lines	Code	What it does
6–32	DocumentUploadResponse / DocumentResponse / DocumentListResponse	Upload response includes status and message. DocumentResponse includes full metadata. List wraps an array + total.

6.12 Pydantic Schemas — Query / Chat

query.py

 File: backend/app/schemas/query.py

 **Purpose:** Models for chat requests, responses, and history.

Full Source

```
1 from pydantic import BaseModel
2 from typing import Optional
3 from datetime import datetime
4
5
6 class ChatRequest(BaseModel):
7     query: str
8
9
10 class ChatResponse(BaseModel):
11     response: str
12     citations: list[dict] = []
13     evaluation: dict = {}
14     retry_count: int = 0
15     latency_ms: float = 0
16
17
18 class ChatHistoryItem(BaseModel):
19     id: int
20     query: str
21     response: str
22     citations_json: Optional[list[dict]] = None
23     created_at: datetime
24
25     class Config:
26         from_attributes = True
27
28
29 class ChatHistoryResponse(BaseModel):
30     messages: list[ChatHistoryItem]
31     total: int
32
```

Line-by-Line Explanation

Lines	Code	What it does
6–7	<code>class ChatRequest(BaseModel): query: str</code>	Simple input: just the question text.
10–31	<code>ChatResponse / ChatHistoryItem / ChatHistoryResponse</code>	ChatResponse: response + citations + evaluation + retry_count + latency_ms. ChatHistoryItem adds id and created_at timestamp.

6.12 Pydantic Schemas — Study Plan

planner.py

 File: backend/app/schemas/planner.py

 **Purpose:** Models for study plan generation and retrieval.

Full Source

```
1 from pydantic import BaseModel
2 from typing import Optional
3 from datetime import datetime
4
5
6 class StudyPlanRequest(BaseModel):
7     days: int = 14
8     hours_per_day: float = 4.0
9
10
11 class TopicPriority(BaseModel):
12     topic: str
13     priority: str # high/medium/low
14     estimated_hours: float
15     pyq_frequency: int = 0
16     rationale: str = ""
17
18
19 class StudyPlanResponse(BaseModel):
20     id: int
21     course_id: int
22     plan_data: dict
23     created_at: datetime
24
25     class Config:
26         from_attributes = True
27
```

Line-by-Line Explanation

Lines	Code	What it does
6–26	StudyPlanRequest / StudyPlanResponse	Request: days (default 14) + hours_per_day (default 4.0). Response: id, course_id, plan_data (JSON), created_at.

6.13 Frontend Entry & Routing — main.jsx

main.jsx

 File: frontend/src/main.jsx

📌 **Purpose:** React entry point. Wraps the entire app in AuthProvider (global auth state) and BrowserRouter (client-side routing).

Full Source

```
1 import React from 'react'
2 import ReactDOM from 'react-dom/client'
3 import { BrowserRouter } from 'react-router-dom'
4 import App from './App.jsx'
5 import { AuthProvider } from './context/AuthContext'
6 import 'bootstrap/dist/css/bootstrap.min.css'
7
8 ReactDOM.createRoot(document.getElementById('root')).render(
9   <React.StrictMode>
10     <AuthProvider>
11       <BrowserRouter>
12         <App />
13       </BrowserRouter>
14     </AuthProvider>
15   </React.StrictMode>,
16 )
17
```

Line-by-Line Explanation

Lines	Code	What it does
1–5	import React, ReactDOM, BrowserRouter, App, AuthProvider	Imports the React library, DOM renderer, router, root App component, and the auth context provider.
8–16	ReactDOM.createRoot(document.getElementById('root')).render(<React.StrictMode>...)	Mounts the React app into the #root div. StrictMode enables extra dev warnings. AuthProvider → BrowserRouter → App establishes the component hierarchy.

6.13 Frontend Entry & Routing — App.jsx

App.jsx

📄 File: frontend/src/App.jsx

📌 **Purpose:** Root component: defines the navbar and all client-side routes. ProtectedRoute redirects unauthenticated users to /login.

Full Source

```
1 import React, { useContext } from 'react';
2 import { Container, Navbar, Nav, Button } from 'react-bootstrap';
3 import { Routes, Route, Navigate, Link, useLocation } from 'react-router-dom';
4 import Login from './pages/Login';
5 import Register from './pages/Register';
6 import Dashboard from './pages/Dashboard';
7 import CourseDetail from './pages/CourseDetail';
8 import { AuthContext } from './context/AuthContext';
9
10 const ProtectedRoute = ({ children }) => {
11   const { user, loading } = useContext(AuthContext);
12   if (loading) return <div className="text-center mt-5">Loading...</div>;
13   return user ? children : <Navigate to="/login" />;
14 };
15
16 function App() {
17   const { user, logout } = useContext(AuthContext);
18   const location = useLocation();
19
20   return (
21     <>
22       <Navbar bg="dark" variant="dark" expand="lg" className="shadow">
23         <Container>
24           <Navbar.Brand as={Link} to="/" className="fw-bold">
25             📖 ExamGPT
26           </Navbar.Brand>
27           <Navbar.Toggle aria-controls="main-nav" />
28           <Navbar.Collapse id="main-nav">
29             <Nav className="me-auto">
30               {user && (
31                 <Nav.Link as={Link} to="/dashboard" active={location.pathname ===
'/dashboard'}>
32                   📚 Courses
33                 </Nav.Link>
34               )}
35             </Nav>
36             <Nav>
37               {user ? (
38                 <>
39                   <Navbar.Text className="me-3">
40                     👤 {user.name || user.username}
41                   </Navbar.Text>
42                   <Button variant="outline-light" size="sm"
onClick={logout}>Logout</Button>
43                 </>
44               ) : (
45                 <>
46                   <Nav.Link as={Link} to="/login">Login</Nav.Link>
47                   <Nav.Link as={Link} to="/register">Register</Nav.Link>
48                 </>
49               )}
50             </Nav>
51           </Navbar.Collapse>

```

```

52         </Container>
53     </Navbar>
54
55     <Routes>
56         <Route path="/login" element={<Login />} />
57         <Route path="/register" element={<Register />} />
58         <Route path="/dashboard" element={
59             <ProtectedRoute><Dashboard /></ProtectedRoute>
60         } />
61         <Route path="/courses/:courseId" element={
62             <ProtectedRoute><CourseDetail /></ProtectedRoute>
63         } />
64         <Route path="/" element={<Navigate to={user ? "/dashboard" : "/login"} />} />
65         <Route path="*" element={<Navigate to="/" />} />
66     </Routes>
67 </>
68 );
69 }
70
71 export default App;
72

```

Line-by-Line Explanation

Lines	Code	What it does
10–14	const ProtectedRoute = ({ children }) => {...}	Checks if the user is logged in via AuthContext. Shows 'Loading...' during auth check. Redirects to /login if not authenticated.
16–67	function App() {...}	Renders the Bootstrap navbar (brand, courses link, user info + logout button) and defines routes: /login, /register, /dashboard (protected), /courses/:courseId (protected), / (redirect), * (catch-all).

6.13 Frontend Entry & Routing — Vite Config

vite.config.js

 File: frontend/vite.config.js

 **Purpose:** Vite build configuration for the React frontend.

Full Source

```

1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vitejs.dev/config/

```



```

5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     port: 3000,
9     host: true
10  }
11 })
12

```

Line-by-Line Explanation

Lines	Code	What it does
1–11	export default defineConfig({...})	Registers the React plugin for JSX. Dev server on port 3000 with host:true (needed inside Docker containers).

6.13 Global State & API Client — AuthContext.jsx

AuthContext.jsx

 File: frontend/src/context/AuthContext.jsx

 **Purpose:** React Context providing global authentication state. Any component can access user, login, register, logout without prop drilling.

Full Source

```

1  import React, { createContext, useState, useEffect } from 'react';
2  import api from '../services/api';
3
4  export const AuthContext = createContext();
5
6  export const AuthProvider = ({ children }) => {
7    const [user, setUser] = useState(null);
8    const [loading, setLoading] = useState(true);
9
10   useEffect(() => {
11     const fetchUser = async () => {
12       const token = localStorage.getItem('token');
13       if (token) {
14         try {
15           const res = await api.get('/auth/me');
16           setUser(res.data);
17         } catch (error) {
18           console.error("Failed to fetch user", error);
19           localStorage.removeItem('token');
20         }
21       }
22       setLoading(false);
23     };
24
25     fetchUser();
26   }, []);
27

```

```

28   const login = async (username, password) => {
29     const formData = new URLSearchParams();
30     formData.append('username', username);
31     formData.append('password', password);
32
33     const res = await api.post('/auth/login', formData, {
34       headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
35     });
36     localStorage.setItem('token', res.data.access_token);
37
38     // Fetch user details
39     const userRes = await api.get('/auth/me');
40     setUser(userRes.data);
41   };
42
43   const register = async (username, email, password, name) => {
44     await api.post('/auth/register', { username, email, password, name });
45   };
46
47   const logout = () => {
48     localStorage.removeItem('token');
49     setUser(null);
50   };
51
52   return (
53     <AuthContext.Provider value={{ user, login, register, logout, loading }}>
54       {children}
55     </AuthContext.Provider>
56   );
57 };
58

```

Line-by-Line Explanation

Lines	Code	What it does
6–26	const AuthProvider = ({ children }) => {...}	On mount: checks localStorage for a JWT token, validates it via /auth/me, sets user state. Handles expired/invalid tokens.
28–50	login / register / logout functions	login: sends form-encoded credentials, saves JWT, fetches user details. register: POSTs to /auth/register. logout: removes JWT + clears user state.

6.13 Global State & API Client — api.js

api.js

 File: frontend/src/services/api.js

📌 **Purpose:** Axios HTTP client with JWT interceptors and pre-built API wrappers for every endpoint.

Full Source

```
1  import axios from 'axios';
2
3  const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8001';
4
5  const api = axios.create({
6    baseURL: `${API_URL}/api/v1`,
7  });
8
9  // Attach JWT token to every request
10 api.interceptors.request.use(
11   (config) => {
12     const token = localStorage.getItem('token');
13     if (token) {
14       config.headers.Authorization = `Bearer ${token}`;
15     }
16     return config;
17   },
18   (error) => Promise.reject(error)
19 );
20
21 // Handle 401 responses (expired token)
22 api.interceptors.response.use(
23   (response) => response,
24   (error) => {
25     if (error.response?.status === 401) {
26       localStorage.removeItem('token');
27       window.location.href = '/login';
28     }
29     return Promise.reject(error);
30   }
31 );
32
33 // — Auth —————
34 export const authAPI = {
35   login: (username, password) => {
36     const formData = new URLSearchParams();
37     formData.append('username', username);
38     formData.append('password', password);
39     return api.post('/auth/login', formData, {
40       headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
41     });
42   },
43   register: (data) => api.post('/auth/register', data),
44   me: () => api.get('/auth/me'),
45 };
46
47 // — Courses —————
48 export const coursesAPI = {
49   list: () => api.get('/courses/'),
50   get: (id) => api.get(`/courses/${id}`),
51   create: (data) => api.post('/courses/', data),
52   delete: (id) => api.delete(`/courses/${id}`),
53 };
```

```

54
55 // — Documents —————
56 export const documentsAPI = {
57   list: (courseId) => api.get(`/courses/${courseId}/documents`),
58   upload: (courseId, file, docType = 'notes') => {
59     const formData = new FormData();
60     formData.append('file', file);
61     formData.append('doc_type', docType);
62     return api.post(`/courses/${courseId}/documents`, formData, {
63       headers: { 'Content-Type': 'multipart/form-data' },
64     });
65   },
66 };
67
68 // — Query —————
69 export const queryAPI = {
70   ask: (courseId, query) => api.post(`/courses/${courseId}/query`, { query }),
71   history: (courseId) => api.get(`/courses/${courseId}/chat-history`),
72 };
73
74 // — Study Plan —————
75 export const studyPlanAPI = {
76   generate: (courseId, data) => api.post(`/courses/${courseId}/study-plan`, data),
77   get: (courseId) => api.get(`/courses/${courseId}/study-plan`),
78 };
79
80 export default api;
81

```

Line-by-Line Explanation

Lines	Code	What it does
1–7	const api = axios.create({baseUrl: ...})	Creates an Axios instance with the backend URL (from VITE_API_URL env var) as the base URL.
9–31	Request & Response interceptors	Request interceptor: auto-attaches JWT from localStorage to every outgoing request. Response interceptor: on 401, removes token and redirects to /login.
33–78	authAPI / coursesAPI / documentsAPI / queryAPI / studyPlanAPI	Pre-built API wrappers organized by feature. Each returns an Axios promise. documentsAPI.upload uses FormData for multipart file upload.

6.14 Pages — Login

Login.jsx

 File: frontend/src/pages/Login.jsx

 **Purpose:** Login page with username/password form, error handling, and link to register.

Full Source

```
1  import React, { useState, useContext } from 'react';
2  import { Form, Button, Card, Container, Alert } from 'react-bootstrap';
3  import { AuthContext } from '../context/AuthContext';
4  import { useNavigate, Link } from 'react-router-dom';
5
6  export default function Login() {
7    const [username, setUsername] = useState('');
8    const [password, setPassword] = useState('');
9    const [error, setError] = useState(null);
10   const { login } = useContext(AuthContext);
11   const navigate = useNavigate();
12
13   const handleSubmit = async (e) => {
14     e.preventDefault();
15     try {
16       await login(username, password);
17       navigate('/chat');
18     } catch (err) {
19       setError('Invalid username or password');
20     }
21   };
22
23   return (
24     <Container className="d-flex justify-content-center align-items-center mt-5">
25       <Card style={{ width: '400px' }} className="shadow">
26         <Card.Body>
27           <h3 className="text-center mb-4">Login</h3>
28           {error && <Alert variant="danger">{error}</Alert>}
29           <Form onSubmit={handleSubmit}>
30             <Form.Group className="mb-3">
31               <Form.Label>Username</Form.Label>
32               <Form.Control type="text" value={username} onChange={e =>
33setUsername(e.target.value)} required />
34             </Form.Group>
35             <Form.Group className="mb-3">
36               <Form.Label>Password</Form.Label>
37               <Form.Control type="password" value={password} onChange={e =>
38setPassword(e.target.value)} required />
39             </Form.Group>
40             <Button variant="primary" type="submit" className="w-100">Login</Button>
41             <div className="text-center mt-3">
42               Don't have an account? <Link to="/register">Register</Link>
43             </div>
44           </Card.Body>
45         </Card>
46       </Container>
```

```
47 }  
48
```

Line-by-Line Explanation

Lines	Code	What it does
6–11	<pre>const [username, setUsername] = useState("") const { login } = useContext(AuthContext)</pre>	React state for form fields + error. Gets the login function from AuthContext.
13–21	<pre>const handleSubmit = async (e) => {...}</pre>	Calls login(username, password). On success: navigates to /chat. On failure: shows 'Invalid username or password'.
23–47	<pre>return (<Container>...<Card>...<Form>...)</pre>	Centered Bootstrap Card with username/password inputs, submit button, and 'Don't have an account? Register' link.

6.14 Pages — Register

Register.jsx

 File: frontend/src/pages/Register.jsx

 **Purpose:** Registration page with name, username, email, and password fields.

Full Source

```
1 import React, { useState, useContext } from 'react';  
2 import { Form, Button, Card, Container, Alert } from 'react-bootstrap';  
3 import { AuthContext } from '../context/AuthContext';  
4 import { useNavigate, Link } from 'react-router-dom';  
5  
6 export default function Register() {  
7   const [formData, setFormData] = useState({ username: '', email: '', password: '',  
name: '' });  
8   const [error, setError] = useState(null);  
9   const { register } = useContext(AuthContext);  
10  const navigate = useNavigate();  
11  
12  const handleChange = (e) => {  
13    setFormData({...formData, [e.target.name]: e.target.value});  
14  }  
15  
16  const handleSubmit = async (e) => {  
17    e.preventDefault();  
18    try {  
19      await register(formData.username, formData.email, formData.password,  
formData.name);  
20      navigate('/login');
```

```

21     } catch (err) {
22         setError('Registration failed. Please check details.');
```

```

23     }
24 };
25
26 return (
27     <Container className="d-flex justify-content-center align-items-center mt-5">
28         <Card style={{ width: '400px' }} className="shadow">
29             <Card.Body>
30                 <h3 className="text-center mb-4">Register</h3>
31                 {error && <Alert variant="danger">{error}</Alert>}
32                 <Form onSubmit={handleSubmit}>
33                     <Form.Group className="mb-3">
34                         <Form.Label>Name</Form.Label>
35                         <Form.Control type="text" name="name" onChange={handleChange} required
/>
36                     </Form.Group>
37                     <Form.Group className="mb-3">
38                         <Form.Label>Username</Form.Label>
39                         <Form.Control type="text" name="username" onChange={handleChange}
required />
40                     </Form.Group>
41                     <Form.Group className="mb-3">
42                         <Form.Label>Email</Form.Label>
43                         <Form.Control type="email" name="email" onChange={handleChange} required
/>
44                     </Form.Group>
45                     <Form.Group className="mb-3">
46                         <Form.Label>Password</Form.Label>
47                         <Form.Control type="password" name="password" onChange={handleChange}
required />
48                     </Form.Group>
49                     <Button variant="success" type="submit" className="w-
100">Register</Button>
50                 </Form>
51                 <div className="text-center mt-3">
52                     Already have an account? <Link to="/login">Login</Link>
53                 </div>
54             </Card.Body>
55         </Card>
56     </Container>
57 );
58 }
59
```

Line-by-Line Explanation

Lines	Code	What it does
6–24	State + handleChange + handleSubmit	Single formData state object with handleChange updating by field name. On success: redirects to /login.
26–58	JSX	Centered Bootstrap Card with 4 input fields, submit button, and login link.

6.14 Pages — Dashboard

Dashboard.jsx

 File: frontend/src/pages/Dashboard.jsx

 **Purpose:** Course management dashboard showing a responsive grid of course cards with create/delete functionality.

Full Source

```
1  import React, { useState, useEffect } from 'react';
2  import { Container, Row, Col, Card, Button, Modal, Form, Badge, Spinner, Alert } from
'react-bootstrap';
3  import { useNavigate } from 'react-router-dom';
4  import { coursesAPI } from '../services/api';
5
6  export default function Dashboard() {
7    const [courses, setCourses] = useState([]);
8    const [loading, setLoading] = useState(true);
9    const [error, setError] = useState('');
10   const [showModal, setShowModal] = useState(false);
11   const [newCourse, setNewCourse] = useState({ name: '', description: '' });
12   const [creating, setCreating] = useState(false);
13   const navigate = useNavigate();
14
15   const fetchCourses = async () => {
16     try {
17       const res = await coursesAPI.list();
18       setCourses(res.data.courses);
19     } catch (err) {
20       setError('Failed to load courses');
21     } finally {
22       setLoading(false);
23     }
24   };
25
26   useEffect(() => { fetchCourses(); }, []);
27
28   const handleCreate = async (e) => {
29     e.preventDefault();
30     setCreating(true);
31     try {
32       await coursesAPI.create(newCourse);
33       setShowModal(false);
34       setNewCourse({ name: '', description: '' });
35       fetchCourses();
36     } catch (err) {
37       setError('Failed to create course');
38     } finally {
39       setCreating(false);
40     }
41   };
42
43   const handleDelete = async (id) => {
44     if (!window.confirm('Delete this course and all its data?')) return;
45     try {
```



```

46     await coursesAPI.delete(id);
47     fetchCourses();
48   } catch (err) {
49     setError('Failed to delete course');
50   }
51   };
52
53   if (loading) return <div className="text-center mt-5"><Spinner animation="border"
/></div>;
54
55   return (
56     <Container className="py-4">
57       <div className="d-flex justify-content-between align-items-center mb-4">
58         <div>
59           <h2><img alt="book icon" data-bbox="298 288 312 302"/> My Courses</h2>
60           <p className="text-muted">Manage your courses and study materials</p>
61         </div>
62         <Button variant="primary" size="lg" onClick={() => setShowModal(true)}>
63           + New Course
64         </Button>
65       </div>
66
67       {error && <Alert variant="danger" onClose={() => setError('')}
dismissible>{error}</Alert>}
68
69       {courses.length === 0 ? (
70         <Card className="text-center p-5">
71           <Card.Body>
72             <h4>No courses yet</h4>
73             <p className="text-muted">Create your first course to start uploading
materials and studying.</p>
74             <Button variant="primary" onClick={() => setShowModal(true)}>Create
Course</Button>
75           </Card.Body>
76         </Card>
77       ) : (
78         <Row xs={1} md={2} lg={3} className="g-4">
79           {courses.map(course => (
80             <Col key={course.id}>
81               <Card className="h-100 shadow-sm" style={{ cursor: 'pointer' }}>
82                 <Card.Body onClick={() => navigate(`/courses/${course.id}`)}>
83                   <Card.Title>{course.name}</Card.Title>
84                   <Card.Text className="text-muted">
85                     {course.description || 'No description'}
86                   </Card.Text>
87                   <Badge bg="info">{course.document_count} documents</Badge>
88                 </Card.Body>
89                 <Card.Footer className="d-flex justify-content-between">
90                   <small className="text-muted">
91                     Created {new Date(course.created_at).toLocaleDateString()}
92                   </small>
93                   <Button variant="outline-danger" size="sm" onClick={(e) => {
e.stopPropagation(); handleDelete(course.id); }}>
94                     Delete
95                   </Button>
96                 </Card.Footer>
97             </Col>
98           )}

```

```

99         )))
100     </Row>
101 })
102
103 { /* Create Course Modal */
104 <Modal show={showModal} onHide={() => setShowModal(false)}>
105     <Modal.Header closeButton>
106         <Modal.Title>Create New Course</Modal.Title>
107     </Modal.Header>
108     <Form onSubmit={handleCreate}>
109         <Modal.Body>
110             <Form.Group className="mb-3">
111                 <Form.Label>Course Name</Form.Label>
112                 <Form.Control
113                     type="text"
114                     placeholder="e.g., Data Structures & Algorithms"
115                     value={newCourse.name}
116                     onChange={e => setNewCourse({ ...newCourse, name: e.target.value })}
117                     required
118                 />
119             </Form.Group>
120             <Form.Group>
121                 <Form.Label>Description (optional)</Form.Label>
122                 <Form.Control
123                     as="textarea"
124                     rows={3}
125                     placeholder="Course description..."
126                     value={newCourse.description}
127                     onChange={e => setNewCourse({ ...newCourse, description:
e.target.value })}
128                 />
129             </Form.Group>
130         </Modal.Body>
131         <Modal.Footer>
132             <Button variant="secondary" onClick={() =>
setShowModal(false)}>Cancel</Button>
133             <Button variant="primary" type="submit" disabled={creating}>
134                 {creating ? <Spinner size="sm" /> : 'Create Course'}
135             </Button>
136         </Modal.Footer>
137     </Form>
138 </Modal>
139 </Container>
140 );
141 }
142

```

Line-by-Line Explanation

Lines	Code	What it does
15–24	const fetchCourses = async () => {...}	Calls coursesAPI.list() and stores courses in state.
28–51	handleCreate / handleDelete	Create: API call + refresh list. Delete: confirmation dialog + API call.
55–141	JSX: header, empty state, course grid, create modal	Responsive grid (1/2/3 cols). Each card shows name,

		description, document count, date, delete button. Modal form for creating courses.
--	--	--

6.14 Pages — CourseDetail

CourseDetail.jsx

 File: frontend/src/pages/CourseDetail.jsx

 **Purpose:** Course detail page with three tabs: Chat, Documents, and Study Plan.

Full Source

```

1  import React, { useState, useEffect } from 'react';
2  import { Container, Row, Col, Card, Tab, Tabs, Badge, Spinner, Alert, Button } from
'react-bootstrap';
3  import { useParams, useNavigate } from 'react-router-dom';
4  import { coursesAPI, documentsAPI } from '../services/api';
5  import DocumentUpload from '../components/DocumentUpload';
6  import ChatInterface from '../components/ChatInterface';
7  import StudyPlanView from './StudyPlan';
8
9  export default function CourseDetail() {
10   const { courseId } = useParams();
11   const navigate = useNavigate();
12   const [course, setCourse] = useState(null);
13   const [documents, setDocuments] = useState([]);
14   const [loading, setLoading] = useState(true);
15   const [activeTab, setActiveTab] = useState('chat');
16
17   const fetchCourse = async () => {
18     try {
19       const res = await coursesAPI.get(courseId);
20       setCourse(res.data);
21     } catch (err) {
22       navigate('/dashboard');
23     }
24   };
25
26   const fetchDocuments = async () => {
27     try {
28       const res = await documentsAPI.list(courseId);
29       setDocuments(res.data.documents);
30     } catch (err) {
31       console.error('Failed to load documents');
32     }
33   };
34
35   useEffect(() => {
36     Promise.all([fetchCourse(), fetchDocuments()]).then(() => setLoading(false));
37   }, [courseId]);
38
39   if (loading) return <div className="text-center mt-5"><Spinner animation="border"
/></div>;
40   if (!course) return <Alert variant="danger">Course not found</Alert>;

```

```

41
42   const statusColor = { uploaded: 'secondary', processing: 'warning', ready:
'success', failed: 'danger' };
43
44   return (
45     <Container className="py-4">
46       <div className="d-flex justify-content-between align-items-center mb-4">
47         <div>
48           <Button variant="link" className="p-0 mb-2" onClick={() =>
navigate('/dashboard')}>
49             ← Back to Courses
50           </Button>
51           <h2>{course.name}</h2>
52           <p className="text-muted">{course.description || 'No description'}</p>
53         </div>
54         <Badge bg="info" className="fs-6">{documents.length} documents</Badge>
55       </div>
56
57       <Tabs activeKey={activeTab} onSelect={setActiveTab} className="mb-4">
58         <Tab eventKey="chat" title="💬 Chat">
59           <ChatInterface courseId={courseId} />
60         </Tab>
61
62         <Tab eventKey="documents" title={`📄 Documents (${documents.length})`>
63           <Row className="mb-4">
64             <Col>
65               <DocumentUpload courseId={courseId} onUploadComplete={() => {
fetchDocuments(); fetchCourse(); }} />
66             </Col>
67           </Row>
68           {documents.length === 0 ? (
69             <Alert variant="info">No documents uploaded yet. Upload your syllabus,
notes, or PYQs above.</Alert>
70           ) : (
71             <Row xs={1} md={2} className="g-3">
72               {documents.map(doc => (
73                 <Col key={doc.id}>
74                   <Card>
75                     <Card.Body>
76                       <div className="d-flex justify-content-between">
77                         <div>
78                           <Card.Title className="fs-6">{doc.filename}</Card.Title>
79                           <Badge bg="primary" className="me-2">{doc.doc_type}</Badge>
80                           <Badge bg={statusColor[doc.status] ||
'secondary'}>{doc.status}</Badge>
81                         </div>
82                       </div>
83                       <small className="text-muted">
84                         Uploaded {new Date(doc.created_at).toLocaleDateString()}
85                       </small>
86                     </Card.Body>
87                   </Card>
88                 </Col>
89               )]}
90             </Row>
91           )}
92         </Tab>
93

```

```

94         <Tab eventKey="studyplan" title="📅 Study Plan">
95             <StudyPlanView courseId={courseId} />
96         </Tab>
97     </Tabs>
98 </Container>
99 );
100 }
101

```

Line-by-Line Explanation

Lines	Code	What it does
9–37	Data fetching with Promise.all	Fetches course details and document list in parallel on mount.
44–100	Tabs: Chat / Documents / Study Plan	Chat tab: ChatInterface component. Documents tab: upload form + document list with status badges. Study Plan tab: StudyPlanView component.

6.14 Pages — Study Plan View

StudyPlan.jsx

📁 File: frontend/src/pages/StudyPlan.jsx

📌 **Purpose:** Study plan generator and viewer with configurable days/hours.

Full Source

```

1  import React, { useState, useEffect } from 'react';
2  import { Card, Button, Form, Spinner, Alert, Badge } from 'react-bootstrap';
3  import { studyPlanAPI } from '../services/api';
4
5  export default function StudyPlanView({ courseId }) {
6      const [plan, setPlan] = useState(null);
7      const [loading, setLoading] = useState(false);
8      const [generating, setGenerating] = useState(false);
9      const [error, setError] = useState('');
10     const [days, setDays] = useState(14);
11     const [hoursPerDay, setHoursPerDay] = useState(4);
12
13     const fetchPlan = async () => {
14         setLoading(true);
15         try {
16             const res = await studyPlanAPI.get(courseId);
17             setPlan(res.data.plan_data);
18         } catch (err) {
19             // No plan yet – that's okay
20             setPlan(null);
21         } finally {
22             setLoading(false);
23         }
24     }
25
26     useEffect(() => {
27         fetchPlan();
28     }, [courseId]);
29
30     return (
31         <div>
32             <h3>Study Plan</h3>
33             <Form>
34                 <div>
35                     <div>Days</div>
36                     <div>Hours per day</div>
37                 </div>
38                 <Button type="button" onClick={fetchPlan}>Generate</Button>
39             </Form>
40             <div>
41                 <div>Plan</div>
42                 <div>
43                     <div>Loading</div>
44                     <div>Error</div>
45                 </div>
46             </div>
47         </div>
48     );
49 }

```

```

24   };
25
26   useEffect(() => { fetchPlan(); }, [courseId]);
27
28   const handleGenerate = async (e) => {
29     e.preventDefault();
30     setGenerating(true);
31     setError('');
32     try {
33       const res = await studyPlanAPI.generate(courseId, { days, hours_per_day:
hoursPerDay });
34       setPlan(res.data.plan_data);
35     } catch (err) {
36       setError('Failed to generate study plan. Make sure you have uploaded documents
first.');
```

first.');

```

37     } finally {
38       setGenerating(false);
39     }
40   };
41
42   if (loading) return <div className="text-center"><Spinner animation="border"
/></div>;
43
44   return (
45     <div>
46       {/* Generation Form */}
47       <Card className="mb-4">
48         <Card.Body>
49           <Card.Title>Generate Study Plan</Card.Title>
50           <Form onSubmit={handleGenerate} className="d-flex gap-3 align-items-end">
51             <Form.Group>
52               <Form.Label>Days until exam</Form.Label>
53               <Form.Control type="number" value={days} onChange={e =>
setDays(Number(e.target.value))} min={1} max={90} />
54             </Form.Group>
55             <Form.Group>
56               <Form.Label>Hours/day</Form.Label>
57               <Form.Control type="number" value={hoursPerDay} onChange={e =>
setHoursPerDay(Number(e.target.value))} min={1} max={16} step={0.5} />
58             </Form.Group>
59             <Button type="submit" variant="primary" disabled={generating}>
60               {generating ? <><Spinner size="sm" /> Generating...</> : '🌀 Generate
Plan'}
61             </Button>
62           </Form>
63         </Card.Body>
64       </Card>
65
66       {error && <Alert variant="danger">{error}</Alert>}
67
68       {/* Display Plan */}
69       {plan && (
70         <div>
71           <h4>{plan.title || 'Your Study Plan'}</h4>
72           <p className="text-muted">
73             {plan.total_days} days • {plan.hours_per_day} hours/day • {plan.total_days
* plan.hours_per_day} total hours
74           </p>

```

```

75
76     {plan.priority_topics?.length > 0 && (
77         <Card className="mb-3">
78             <Card.Body>
79                 <Card.Title>🔥 Priority Topics</Card.Title>
80                 <div className="d-flex flex-wrap gap-2">
81                     {plan.priority_topics.map((t, i) => (
82                         <Badge key={i} bg="danger" className="fs-6">{t}</Badge>
83                     ))}
84                 </div>
85             </Card.Body>
86         </Card>
87     )}
88
89     {plan.phases?.map((phase, i) => (
90         <Card key={i} className="mb-3">
91             <Card.Header>
92                 <strong>Phase {phase.phase}: {phase.name}</strong>
93                 <Badge bg="secondary" className="ms-2">{phase.days}</Badge>
94                 <Badge bg="info" className="ms-2">{phase.hours}h</Badge>
95             </Card.Header>
96             <Card.Body>
97                 <p><strong>Focus:</strong> {phase.focus}</p>
98                 {phase.topics?.length > 0 && (
99                     <div className="mb-2">
100                         <strong>Topics:</strong>{' '}
101                         {phase.topics.map((t, j) => <Badge key={j} bg="primary"
className="me-1">{t}</Badge>)}
102                     </div>
103                     )}
104                 {phase.activities?.length > 0 && (
105                     <ul className="mb-0">
106                         {phase.activities.map((a, j) => <li key={j}>{a}</li>)}
107                     </ul>
108                     )}
109             </Card.Body>
110         </Card>
111     )}}
112
113     {plan.tips?.length > 0 && (
114         <Card className="mb-3">
115             <Card.Body>
116                 <Card.Title>💡 Tips</Card.Title>
117                 <ul className="mb-0">
118                     {plan.tips.map((t, i) => <li key={i}>{t}</li>)}
119                 </ul>
120             </Card.Body>
121         </Card>
122     )}
123 </div>
124 )}
125
126     {!plan && !loading && (
127         <Alert variant="info">No study plan yet. Upload your materials and generate
one above!</Alert>
128     )}
129 </div>
130 );

```

```

131 }
132

```

Line-by-Line Explanation

Lines	Code	What it does
5–26	State + data fetching	State for plan data, form inputs (days, hoursPerDay). Fetches existing plan on mount.
28–40	handleGenerate	Sends days + hours_per_day to the study plan API. Stores the returned plan_data.
44–131	Plan display	Priority topics as red badges. Phase cards with name/days/hours/focus/topics/activities. Tips as bullet list.

6.15 Components — ChatInterface

ChatInterface.jsx

 File: frontend/src/components/ChatInterface.jsx

 **Purpose:** Real-time chat UI with message history, auto-scroll, loading states, and citation display.

Full Source

```

1  import React, { useState, useEffect, useRef } from 'react';
2  import { Card, Form, Button, Spinner, Badge, Alert } from 'react-bootstrap';
3  import { queryAPI } from '../services/api';
4
5  export default function ChatInterface({ courseId }) {
6    const [messages, setMessages] = useState([]);
7    const [query, setQuery] = useState('');
8    const [loading, setLoading] = useState(false);
9    const [loadingHistory, setLoadingHistory] = useState(true);
10   const messagesEndRef = useRef(null);
11
12   const scrollToBottom = () => {
13     messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
14   };
15
16   useEffect(() => {
17     const fetchHistory = async () => {
18       try {
19         const res = await queryAPI.history(courseId);
20         const history = res.data.messages.reverse().flatMap(m => [
21           { role: 'user', content: m.query },
22           { role: 'assistant', content: m.response, citations: m.citations_json },
23         ]);
24         setMessages(history);
25       } catch (err) {
26         // No history yet
27       } finally {

```



```

28         setLoadingHistory(false);
29     }
30 };
31 fetchHistory();
32 }, [courseId]);
33
34 useEffect(() => { scrollToBottom(); }, [messages]);
35
36 const handleSubmit = async (e) => {
37     e.preventDefault();
38     if (!query.trim() || loading) return;
39
40     const userMsg = { role: 'user', content: query };
41     setMessages(prev => [...prev, userMsg]);
42     setQuery('');
43     setLoading(true);
44
45     try {
46         const res = await queryAPI.ask(courseId, query);
47         const assistantMsg = {
48             role: 'assistant',
49             content: res.data.response,
50             citations: res.data.citations,
51             evaluation: res.data.evaluation,
52             latency: res.data.latency_ms,
53         };
54         setMessages(prev => [...prev, assistantMsg]);
55     } catch (err) {
56         setMessages(prev => [...prev, {
57             role: 'assistant',
58             content: 'Sorry, something went wrong. Please try again.',
59             error: true,
60         }]);
61     } finally {
62         setLoading(false);
63     }
64 };
65
66 if (loadingHistory) return <div className="text-center"><Spinner animation="border"
/></div>;
67
68 return (
69     <div>
70         { /* Messages */ }
71         <div style={{ height: '500px', overflowY: 'auto', border: '1px solid #dee2e6',
borderRadius: '8px', padding: '16px', marginBottom: '16px', backgroundColor: '#f8f9fa' }}>
72             {messages.length === 0 && (
73                 <div className="text-center text-muted mt-5">
74                     <h5>👉 Ask a question about your course!</h5>
75                     <p>Upload materials first, then ask questions here.</p>
76                 </div>
77             )}
78             {messages.map((msg, i) => (
79                 <div key={i} className={`mb-3 d-flex ${msg.role === 'user' ? 'justify-
content-end' : 'justify-content-start'}`}>
80                     <Card style={{ maxWidth: '80%' }} className={msg.role === 'user' ? 'bg-
primary text-white' : msg.error ? 'bg-danger text-white' : 'bg-white'}>
81                         <Card.Body className="py-2 px-3">

```

```

82         <div style={{ whiteSpace: 'pre-wrap' }}>{msg.content}</div>
83         {msg.citations?.length > 0 && (
84             <div className="mt-2 pt-2 border-top">
85                 <small><strong>Sources:</strong></small>
86                 {msg.citations.map((c, j) => (
87                     <Badge key={j} bg="light" text="dark" className="ms-1">
88                          {c.source} {c.year ? `(${c.year})` : ''}
89                     </Badge>
90                 ))}
91             </div>
92         )}
93         {msg.evaluation?.score && (
94             <div className="mt-1">
95                 <small className="text-muted">
96                     Quality: {msg.evaluation.score}/10 • {Math.round(msg.latency)}ms
97                 </small>
98             </div>
99         )}
100     </Card.Body>
101 </Card>
102 </div>
103 )}
104 {loading && (
105     <div className="d-flex justify-content-start mb-3">
106         <Card style={{ maxWidth: '80%' }}>
107             <Card.Body className="py-2 px-3">
108                 <Spinner size="sm" className="me-2" />
109                 Analyzing your question through 7 AI agents...
110             </Card.Body>
111         </Card>
112     </div>
113 )}
114 <div ref={messagesEndRef} />
115 </div>
116
117 {/* Input */}
118 <Form onSubmit={handleSubmit} className="d-flex gap-2">
119     <Form.Control
120         type="text"
121         placeholder="Ask a question about your course material..."
122         value={query}
123         onChange={e => setQuery(e.target.value)}
124         disabled={loading}
125         size="lg"
126     />
127     <Button type="submit" variant="primary" size="lg" disabled={loading ||
!query.trim()}>
128         {loading ? <Spinner size="sm" /> : 'Send'}
129     </Button>
130 </Form>
131 </div>
132 );
133 }
134

```

Line-by-Line Explanation

Lines	Code	What it does
-------	------	--------------

5–14	State: messages, query, loading, loadingHistory, messagesEndRef	Messages array stores alternating user/assistant objects. useRef for auto-scrolling.
16–32	useEffect: fetchHistory	On mount: fetches previous chat messages, reverses them (newest last), formats as user/assistant pairs.
36–63	handleSubmit	Optimistic UI: adds user message immediately, clears input, sends API request. On success: adds assistant message with citations. On failure: adds error message.
68–133	JSX: scrollable container + input	Chat bubbles: blue (user, right-aligned), white (assistant, left-aligned), red (errors). Citations shown as badges. Quality score + latency displayed. Loading indicator: 'Analyzing through 7 AI agents...'

6.15 Components — DocumentUpload

DocumentUpload.jsx

 File: frontend/src/components/DocumentUpload.jsx

 **Purpose:** PDF upload component with file selection, document type dropdown, and upload progress feedback.

Full Source

```

1  import React, { useState } from 'react';
2  import { Card, Form, Button, Spinner, Alert, ProgressBar } from 'react-bootstrap';
3  import { documentsAPI } from '../services/api';
4
5  export default function DocumentUpload({ courseId, onUploadComplete }) {
6    const [file, setFile] = useState(null);
7    const [docType, setDocType] = useState('notes');
8    const [uploading, setUploading] = useState(false);
9    const [message, setMessage] = useState('');
10   const [error, setError] = useState('');
11
12   const handleUpload = async (e) => {
13     e.preventDefault();
14     if (!file) return;
15
16     setUploading(true);
17     setMessage('');

```

```

18     setError('');
19
20     try {
21         const res = await documentsAPI.upload(courseId, file, docType);
22         setMessage(`✅ ${res.data.filename} uploaded and indexed successfully!`);
23         setFile(null);
24         e.target.reset();
25         if (onUploadComplete) onUploadComplete();
26     } catch (err) {
27         setError(err.response?.data?.detail || 'Upload failed. Please try again.');
```

```

28     } finally {
29         setUploading(false);
30     }
31 };
32
33 return (
34     <Card>
35         <Card.Body>
36             <Card.Title>📄 Upload Document</Card.Title>
37             <Form onSubmit={handleUpload}>
38                 <div className="d-flex gap-3 align-items-end">
39                     <Form.Group className="flex-grow-1">
40                         <Form.Label>PDF File</Form.Label>
41                         <Form.Control
42                             type="file"
43                             accept=".pdf"
44                             onChange={e => setFile(e.target.files[0])}
45                             disabled={uploading}>
46                         />
47                     </Form.Group>
48                     <Form.Group>
49                         <Form.Label>Type</Form.Label>
50                         <Form.Select value={docType} onChange={e => setDocType(e.target.value)}
disabled={uploading}>
51                             <option value="notes">📝 Notes</option>
52                             <option value="syllabus">📖 Syllabus</option>
53                             <option value="pyq">📑 PYQ</option>
54                         </Form.Select>
55                     </Form.Group>
56                     <Button type="submit" variant="success" disabled={!file || uploading}>
57                         {uploading ? <><Spinner size="sm" /> Uploading...</> : 'Upload'}
58                     </Button>
59                 </div>
60             </Form>
61             {message && <Alert variant="success" className="mt-3 mb-0">{message}</Alert>}
62             {error && <Alert variant="danger" className="mt-3 mb-0">{error}</Alert>}
63         </Card.Body>
64     </Card>
65 );
66 }
67

```

Line-by-Line Explanation

Lines	Code	What it does
5–10	State: file, docType, uploading, message, error	Tracks selected file, document type (notes/syllabus/pyq), upload

		status, and success/error messages.
12–30	handleUpload	Creates FormData with file + doc_type. POSTs to upload endpoint. Shows success/error messages. Calls onUploadComplete callback to refresh document list.
33–66	JSX: Card with file input + type dropdown + button	Bootstrap Card with file input (PDF only), type dropdown (Notes/Syllabus/PYQ), Upload button with spinner. Success/error alerts.

7. Deployment Architecture (Free Cloud Tier)

7.1 Services Used

Component	Service	Cost	Notes
Frontend	Netlify	Free	React build (dist/) drag-and-drop deploy
Backend API	Hugging Face Spaces	Free (Docker)	Git push triggers auto-rebuild
PostgreSQL	Supabase	Free tier	500 MB, auto-managed
Redis	Upstash	Free tier	10k commands/day, 256 MB
Neo4j	Neo4j AuraDB	Free tier	50k nodes, 175k relationships
File Storage	Supabase Storage	Free tier	1 GB S3-compatible
LLM	Hugging Face Inference	Free tier	Rate-limited, Llama 3 8B
Embeddings	Hugging Face Inference	Free tier	MiniLM-L6-v2

7.2 Required Environment Variables

- DATABASE_URL — PostgreSQL connection string from Supabase
- REDIS_URL — Redis connection string from Upstash
- NEO4J_URI — Bolt URI from Neo4j AuraDB
- NEO4J_USER — Neo4j username (usually 'neo4j')
- NEO4J_PASSWORD — Neo4j password
- SUPABASE_URL — Supabase project URL
- SUPABASE_KEY — Supabase service role key
- SUPABASE_BUCKET — Storage bucket name (default: examgpt-docs)

- HUGGINGFACE_API_KEY — HF API token
- SECRET_KEY — JWT signing secret (change in production!)
- VITE_API_URL — Backend URL (set in Netlify env vars for frontend)

8. Key Design Decisions

Why LangGraph?

LangGraph gives us a typed, compiled state machine rather than a fragile chain of function calls. Every node reads from and writes to a shared TypedDict, making the data flow explicit and debuggable. The conditional edge (evaluation → retry) would be awkward to implement with plain LangChain.

Why Hybrid Retrieval (FAISS + BM25)?

Semantic search (FAISS) catches paraphrased concepts but can miss exact keyword matches. BM25 catches exact terms but misses semantically similar content. Reciprocal Rank Fusion (RRF) combines both, giving the best of both worlds. The k=60 damping factor is the standard default from the original RRF paper.

Why Supabase instead of raw PostgreSQL?

Supabase provides a free-tier managed PostgreSQL + S3-compatible storage + auth (unused here) in one dashboard. It eliminates DB administration overhead for a student project while still being standard PostgreSQL under the hood.

Why store secrets in environment variables?

12-factor app methodology. Secrets never touch the codebase or version control. Pydantic Settings reads them type-safely. Different environments (dev/staging/prod) just change the env vars.

Why NullPool for SQLAlchemy?

Hugging Face Spaces' firewall aggressively closes idle TCP connections to external databases. Standard connection pooling resulted in 'connection already closed' errors under load. NullPool creates a fresh connection per request and closes it immediately, which is slightly slower but 100% reliable.

Why course_id filtering in the hybrid retriever?

Without it, a student's question about Course A would retrieve document chunks from Course B (data leakage). The course_id_filter parameter was added as a post-retrieval filter to ensure complete course isolation.